

Infra

이 묶음은 `admin_infra`의 config-server가 계정, 홈 디렉터리, Pod, NodePort, Kerberos를 어떻게 다루는지 설명한다. 처음 보는 사람도 어디서 시작해야 할지 바로 잡을 수 있게, 읽는 목적별 경로와 문서 지도를 함께 둔다.

먼저 읽을 길

지금 하려는 일	추천 순서
처음 합류해서 전체를 훑는다	개요 → 시스템 아키텍처 → 운영 매뉴얼
승인 뒤 무슨 작업이 일어나는지 본다	개요 → 시스템 아키텍처 → API 레퍼런스
배포나 장애 대응을 한다	운영 매뉴얼 → 데이터베이스 → Helm 차트 레퍼런스
용어가 낯설다	기초 개념 → 필요한 본문 문서
Kerberos, AD, keytab 흐름을 본다	kdc-setup → 설계 → 운영

문서 지도

묶음	문서	이 문서에서 보는 것	다음에 읽을 문서
입문	개요	config-server가 시스템에서 맡는 역할과 큰 흐름	시스템 아키텍처
설계	시스템 아키텍처	계정, 홈 디렉터리, Pod, NodePort가 만들어지는 순서	데이터베이스, API 레퍼런스
설계	기초 개념	UID/GID, Kubernetes, NFS, NodePort 같은 배경 용어	필요한 본문 문서
설계	데이터베이스	infra-mysql과 NodePort 배정 기록 구조	운영 매뉴얼
작업 시작	처음 작업할 때	브랜치, 로컬 실행, 배포 흐름, PR 규칙	운영 매뉴얼
운영	운영 매뉴얼	배포, 상태 점검, 장애 확인, 복구 순서	데이터베이스, Helm 차트 레퍼런스
운영	API 레퍼런스	config-server 엔드포인트, 입력, 응답, 호출 주체	시스템 아키텍처
운영	Helm 차트 레퍼런스	Helm values와 차트 템플릿이 어디에 쓰이는지	운영 매뉴얼
Kerberos	kdc-setup	AD, Secret, 노드, Pod 사이의 Kerberos 흐름 전체	설계, 운영

이 묶음의 범위

- `infra`는 config-server가 직접 만드는 계정, 홈 디렉터리, Pod, NodePort, Kerberos 준비를 다룬다.

- NAS service keytab, NFS 서버 자체 설정, 사용자 이미지 내부 동작처럼 다른 묶음이 더 적합한 내용은 `system` 문서로 보낸다.
- 실제 값, 비밀번호, keytab, SSH 개인키는 여기서 설명하지 않고 운영 확인 절차와 위치만 적는다.

개요

이 문서에서 다루는 내용

- config-server가 전체 시스템에서 맡는 역할
- 승인 뒤 계정, 홈 디렉터리, Pod, NodePort가 만들어지는 큰 흐름
- 관련 저장소, 운영 주소, 다음에 읽을 문서

빠른 이동

궁금한 내용	바로 볼 문서
승인 뒤 세부 순서가 궁금하다	시스템 아키텍처
Kubernetes, UID/GID, NFS 용어가 낯설다	기초 개념
운영 점검과 장애 대응을 해야 한다	운영 매뉴얼
API 입력과 응답을 확인해야 한다	API 레퍼런스
Kerberos와 AD 설정이 궁금하다	kdc-setup

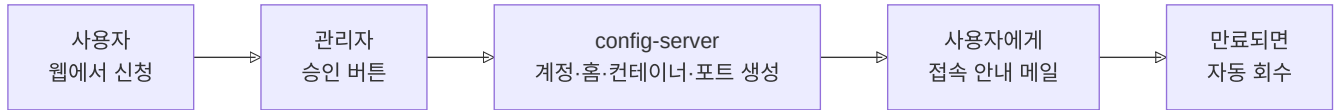
1. config-server가 하는 일

DECS는 연구실 GPU 서버를 웹에서 신청하고 승인받아 사용하는 시스템이다. 세 서버가 다음 일을 나눠 한다.

서버	하는 일
admin_fe	사용자와 관리자가 신청과 승인 결과를 확인하는 화면을 제공한다.
admin_be	신청 내용과 승인 상태를 저장하고 config-server에 작업을 요청한다.
config-server (<code>admin_infra</code>)	계정, 홈 디렉터리, Pod, 접속 포트를 만들거나 지운다.

admin_be가 승인 여부를 정하고, config-server가 Kubernetes와 NAS에 실제 작업을 수행한다. config-server는 스스로 사용자를 승인하지 않는다.

2. 승인 뒤에 일어나는 일



관리자가 승인하면 config-server는 다음 순서로 작업한다.

1. **계정 만들기** — 리눅스 계정(UID)을 공용 계정 파일에 등록
2. **홈 폴더 만들기** — NAS(공용 저장 장비)에 그 사용자의 홈 디렉터리를 생성
3. **GPU Pod 만들기** — GPU 사용량이 낮은 노드를 골라 Pod를 만든다.
4. **접속 포트 열기** — SSH·Jupyter로 접속할 수 있는 포트를 배정한다.
5. **결과 전달** — Pod 이름과 포트를 admin_be에 보내고, admin_be가 사용자에게 안내 메일을 보낸다.

만료되면 Pod와 포트를 지운 뒤 계정과 홈 디렉터리를 정리한다.

3. config-server가 없을 때 하던 일

이 시스템이 없던 때에는 관리자가 서버에 접속해 계정·컨테이너·포트를 모두 명령어로 처리했다.

- 신청이 몰리면 **관리자 한 사람이 병목**이 된다.
- UID 중복·포트 충돌 같은 **수작업 실수**가 발생한다.
- 만료된 자원을 사람이 기억해 삭제해야 하므로 **GPU가 방치**될 수 있다.

config-server는 이 작업을 API로 처리한다. 계정, GPU 노드, 포트가 정해지는 방법은 **시스템 아키텍처**에 있다.

4. 원본 인프라 저장소의 구성

아래 디렉터리는 이 위키가 아닌 `CSID-DGU/admin_infra` 저장소에 있다.

디렉터리	내용
<code>config-server/</code>	Flask API 서버와 Helm 차트
<code>infra-sql/</code>	NodePort 기록을 저장하는 MySQL 배포 파일
<code>pvc-image-chart/</code>	사용자 이미지 파일을 저장하는 PVC 차트

5. 서버 환경 요약

항목	값
배포 위치	FARM Kubernetes, namespace <code>ailab-infra</code>
외부 접근	NodePort 30082 (Service <code>containerssh-config-service</code>)
Swagger UI	<code>http://210.94.179.18:30082/apidocs/</code>
전용 DB (config-server가 관리)	infra-mysql <code>pod_port_db</code> — 데이터베이스

6. 필요한 문서 찾기

상황	추천 페이지
처음 작업한다	이 페이지 → 처음 작업할 때
계정, Pod, GPU 노드, 포트가 어떻게 정해지는지 본다	시스템 아키텍처
API 요청과 응답이 필요하다	API 레퍼런스
점검하거나 장애를 처리한다	운영 매뉴얼
Helm 값이나 차트를 바꾼다	Helm 차트 레퍼런스

시스템 아키텍처

이 문서에서 다루는 내용

- `admin_infra`의 config-server가 계정, 홈 디렉터리, Pod, NodePort를 어떤 순서로 만드는지
- 어떤 정보가 `admin_be`, infra-mysql, `/kube_share`, NAS에 나뉘어 저장되는지
- 계정 생성, Pod 생성, 삭제 흐름에서 어디서 실패할 수 있는지

빠른 이동

필요할 때	문서
실제 운영 명령과 점검 순서를 본다	운영 매뉴얼
요청 형식과 응답을 확인한다	API 레퍼런스

필요할 때	문서
Kerberos 준비 단계를 자세히 본다	kdc-setup

1. 서버마다 하는 일

신청을 받고 승인하는 일, Pod와 계정을 만드는 일, 파일과 인증을 제공하는 일은 서로 다른 서버가 맡는다.

하는 일	서버 또는 서비스	설명
신청과 승인	<code>admin_fe</code> , <code>admin_be</code>	신청을 받고 승인 결과와 사용자 설정을 저장한다.
Pod와 계정 만들기	<code>config-server</code>	계정, 홈 디렉터리, Pod, NodePort Service를 만들거나 지운다.
Pod를 만드는 데 필요한 서비스	Kubernetes, NAS, KDC, Prometheus	컨테이너를 실행하고, 파일을 저장하고, Kerberos 인증과 GPU 노드 정보를 제공한다.

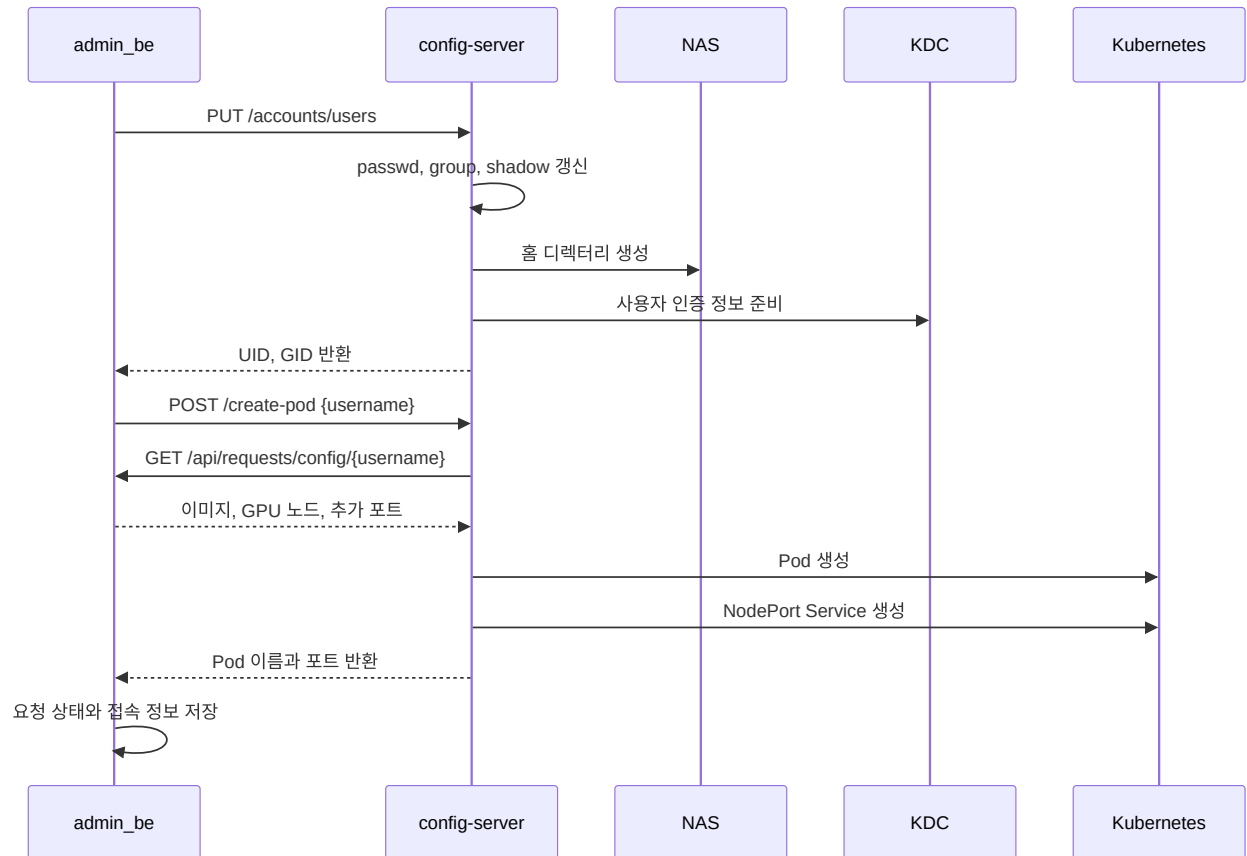
중요한 정보는 다음 위치에 저장한다.

정보	저장 위치	사용하는 곳
신청 상태와 사용자 설정	<code>admin_be</code> DB	<code>admin_be</code> , <code>config-server</code>
NodePort 배정	<code>infra-mysql</code> 의 <code>nodeport_allocations</code>	<code>config-server</code>
UID, GID, 그룹	<code>/kube_share</code> 의 계정 파일	<code>config-server</code> , 사용자 Pod
사용자 홈과 이미지 저장소	NAS	Kubernetes 노드, <code>config-server</code>

`admin_be`는 어떤 사용자에게 Pod를 만들지 정한다. `config-server`는 그 요청을 받아 Pod, 홈 디렉터리, 접속 포트를 만든다. Pod를 만들 때 필요한 이미지, GPU 노드, 추가 포트는 `admin_be`에서 다시 받는다.

2. 승인 후 사용자 환경을 만드는 순서

승인이 끝나면 `admin_be`는 먼저 계정을 만들고, 그다음 Pod를 만든다.



2.1 계정과 홈 디렉터리

`PUT /accounts/users` 는 `/kube_share` 의 `passwd` , `group` , `shadow` 파일에 새 사용자 정보를 쓴다. 새 UID와 GID를 정하는 방법은 [계정 정보](#)에서 확인한다.

홈 디렉터리는 Pod보다 먼저 NAS에 만든다. NFS의 `root_squash` 설정 때문에 Pod 안의 root 사용자는 NAS에 홈 디렉터리를 만들 수 없다. config-server가 NAS에 SSH로 접속해 디렉터리를 만들고, 새 UID와 GID로 소유자를 지정한 뒤 권한을 `700` 으로 설정한다. 홈 생성이 실패하면 계정 파일에 쓴 내용도 되돌리고 계정 생성 요청을 실패로 끝낸다.

2.2 사용자 설정과 Pod 생성

`POST /create-pod` 의 입력은 사용자 이름이다. config-server는 admin_be 의 `GET /api/requests/config/{username}` 을 호출해 다음 값을 가져온다.

- 사용할 이미지
- 사용할 GPU 수와 후보 노드
- 기본 포트 외에 열 포트

config-server는 받은 값으로 Pod를 만든다. 추가 포트에 내부 포트 `6080` 이 있거나 용도가 `novnc` , `vnc` 이면 Pod 환경 변수에 `ENABLE_VNC=true` 도 넣는다. Pod가 접속을 받을 준비가 된 뒤에 NodePort Service를 만들고, Pod 이름과 NodePort 목록을 admin_be 에 돌려준다.

2.3 Pod와 계정 삭제

사용자 환경을 지울 때는 Pod 관련 작업과 계정 관련 작업을 나누어 처리한다.

구분	API	정리 대상
Pod 관련 작업	<code>POST /delete-pod</code>	Pod, NodePort Service, NodePort 배정 기록
계정 관련 작업	<code>DELETE /accounts/users/{username}</code>	계정 파일, NAS 홈, Kerberos 관련 파일

두 요청을 모두 처리해야 계정과 Pod가 함께 삭제된다. 요청 형식과 응답은 [API 레퍼런스](#)에서 확인한다.

3. GPU 노드와 포트 고르는 방식

3.1 노드 선택

config-server는 `admin_be`가 전달한 `gpu_nodes` 목록에서 Pod를 올릴 GPU 노드를 고른다. 노드 이름은 소문자로 바꾸고, 목록에 들어 있던 순서대로 비교한다. `gpu_nodes`가 비어 있으면 Kubernetes에서 준비가 끝난 워커 노드를 가져온다. control-plane 노드에 붙은 `NoSchedule` 표시는 관리용 노드라는 뜻이므로 후보에서 뺀다.

지표	사용 방식
<code>DCGM_FI_DEV_GPU_UTIL</code>	노드 GPU 사용률 평균을 그대로 사용한다.
<code>DCGM_FI_DEV_FB_USED</code>	GPU 메모리 사용량 평균을 GiB 단위로 환산한다.
<code>DCGM_FI_DEV_GPU_TEMP</code>	GPU 온도 평균을 보조 값으로 사용한다.

$$\text{score} = \text{avg}(\text{GPU_UTIL}) + \text{avg}(\text{FB_USED}) / 1024 + \text{avg}(\text{GPU_TEMP}) / 100$$

점수가 가장 낮은 노드를 고른다. 점수가 같으면 후보 목록에서 앞에 있던 노드를 고른다.

각 지표 조회에는 `or vector(0)`이 들어 있다. 어떤 지표가 없으면 그 값은 0으로 계산한다. Prometheus에 연결하지 못했거나 결과가 비어 있으면 그 노드는 고르지 않는다. 모든 후보를 조회하지 못하면 Pod를 만들지 않는다.

고른 노드가 `gpu_nodes` 목록에 있으면 그 항목의 CPU, 메모리, GPU 개수를 Pod 설정에 넣는다. `gpu_nodes` 없이 워커 노드를 찾은 경우에는 config-server에 설정된 기본값을 사용한다.

3.2 NodePort 배정

사용자 Pod에는 SSH(22), Jupyter(8888), `admin_be`가 전달한 추가 포트마다 NodePort Service를 하나씩 만든다. 추가 포트는 받은 순서대로 SSH와 Jupyter 뒤에 붙인다. 어떤 Pod에 어떤 NodePort를 줬는지는 `infra-mysql`의 `nodeport_allocations` 테이블에 적는다.

포트를 고르기 전에는 MySQL 기록과 Kubernetes Service를 비교한다. 이 비교는 마지막 시도 뒤 5분이 지나야 다시 실행되며, 성공하지 못한 시도도 5분 간격에 포함한다. `app=ai lab-nodeport` 라벨과 같은 `pod_name`을 가진 Service가 Kubernetes에 하나도 없고 MySQL에만 남아 있으면 그 Pod의 포트 기록을 지운다. 이 비교는 포트별이 아니라 Pod별로 한다.

포트를 고르고 MySQL에 기록하는 일은 한 번의 DB 작업으로 처리한다. 중간에 실패하면 새 기록은 저장하지 않는다.

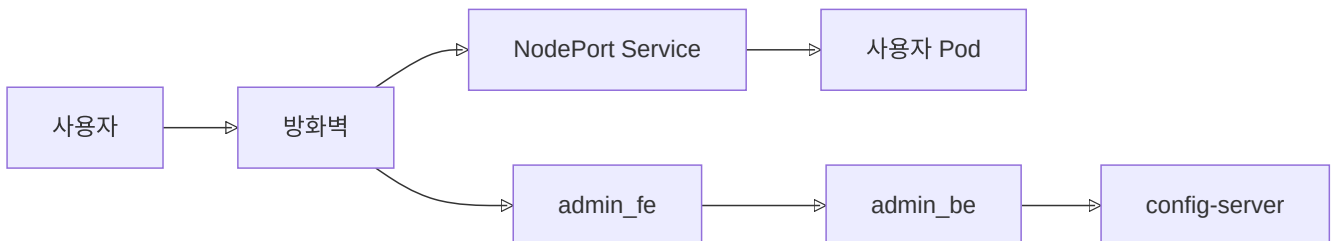
1. `nodeport_allocations`의 포트 기록을 잠가, 동시에 들어온 요청이 같은 번호를 고르지 못하게 한다.
2. MySQL에 적힌 포트와 클러스터 전체 Service가 이미 쓰는 NodePort를 모은다. 클러스터 전체 Service 조회가 실패하면 오류를 로그에 남기고 MySQL 기록만으로 계속 진행한다. 이 경우 다른 Service가 이미 쓰는 포트를 고를 수 있고, 뒤의 Service 생성 단계가 실패할 수 있다.
3. 30000부터 32767까지 오름차순으로 훑어 빈 포트를 필요한 개수만큼 고른다.
4. 고른 포트를 `username`, `pod_name`, `node_name`, 내부 포트, 용도와 함께 MySQL에 저장한다.
5. Pod가 Ready 상태가 되면 기록된 포트마다 NodePort Service를 만든다.

Pod 또는 Service 생성에 실패하면 해당 Pod의 Service, NodePort 배정 행, 생성된 Pod를 정리한다.

구분	저장 위치	의미
요청 포트	<code>admin_be</code> DB의 <code>port_requests</code>	사용자가 요청한 컨테이너 내부 포트
확정 포트	<code>admin_be</code> DB의 <code>pod_external_ports</code>	실제 NodePort와 내부 포트의 연결 정보
배정 기록	<code>infra-mysql</code> 의 <code>nodeport_allocations</code>	Pod와 NodePort Service를 정리할 때 사용하는 기록

3.3 외부 접근 경로

외부 사용자는 방화벽을 거쳐 `admin_fe` 또는 사용자 Pod의 NodePort Service에 접속한다. 방화벽은 외부 포트를 NodePort로 매핑하므로 외부 포트와 NodePort 번호가 같을 필요는 없다. 외부 접속이 되지 않으면 해당 Service의 NodePort와 pfSense 매핑 대상이 같은지 확인한다.



4. 계정, 홈 디렉터리, 인증

4.1 계정 정보 {#account-information}

UID와 GID는 `/kube_share`의 계정 파일에 적힌 값을 사용한다. NAS 파일의 소유자도 이 숫자로 표시되므로, UID와 GID를 다른 데이터베이스에 따로 저장하지 않는다.

계정을 추가할 때는 `passwd` 파일을 잠가, 동시에 들어온 요청이 같은 UID를 고르지 못하게 한다. `/home/`을 쓰는 사용자 중 UID가 20000 이상인 값 가운데 가장 큰 값에 1을 더해 새 UID 후보를 만든다. 이 번호가 시스템 계정을 포함해 이미 쓰이고 있으면, 비어 있는 번호가 나올 때까지 1씩 올린다. 요청에 UID가 들어 있어도 config-server는 사용하지 않고 이 방식으로 정한 값을 반환한다.

새 사용자의 기본 GID는 새 UID와 같은 번호로 정한다. 기본 그룹 이름은 요청의 `primary_group_name`을 쓰고, 이 값이 없으면 사용자 이름을 쓴다. 추가 그룹은 요청에 들어 있는 `{name, gid}` 목록으로 처리한다. 같은 GID의 그룹이 이미 있으면 사용자를 그 그룹에 넣고, 없으면 요청에 적힌 GID로 그룹을 새로 만든다. 추가 그룹의 GID는 config-server가 새

로 정하지 않는다.

`passwd`에 사용자 항목을 쓴 뒤 기본 그룹과 추가 그룹을 `group`에 기록하고, SHA-512 암호 해시를 `shadow`에 기록한다. `sudo` 허용 명령이 설정된 경우에만 사용자별 `sudoers` 파일도 만든다. 이 단계에서 실패하면 앞에서 쓴 계정·그룹 정보를 지운다. NAS 홈 디렉터리의 소유자도 같은 UID와 GID로 바꾼다. Pod를 만들 때 `config-server`는 UID, 기본 GID, 그룹 목록을 환경 변수로 넣는다. 컨테이너가 시작할 때 실행하는 스크립트는 이 값으로 컨테이너 안의 사용자를 만든다. 사용자 Pod는 `/kube_share`를 직접 마운트하지 않는다.

4.2 홈 디렉터리와 이미지 저장소

사용자 홈은 NAS의 NFS 공유를 노드에 마운트한 뒤 Pod의 `/home`에 연결한다. 사용자마다 PVC를 만들지는 않는다. 홈 디렉터리의 소유자와 권한으로 사용자별 접근을 나눈다.

사용자 이미지 tar 파일은 별도의 `pvc-image-store`에 저장한다. `config-server`가 이미지 저장과 로드를 담당하고, Redis에는 이미지 작업의 진행 상태를 기록한다.

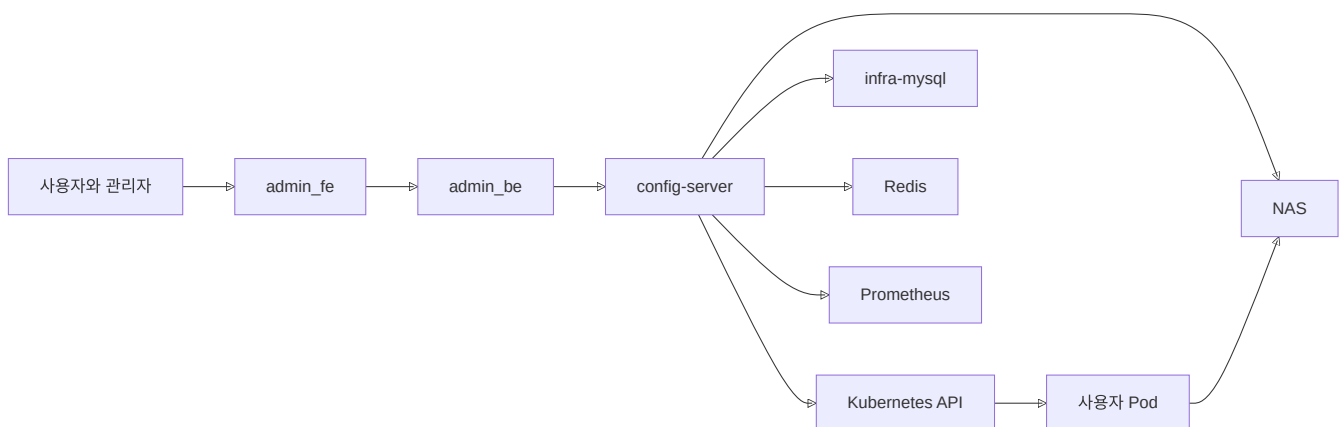
4.3 Kerberos를 사용하는 방식

FARM 환경에서는 NFS 홈을 쓰기 전에 Kerberos 인증을 준비한다. 계정을 만들 때 `config-server`는 사용자 principal을 만들고 keytab을 Kubernetes Secret에 저장한다. Pod를 만들 때는 선택된 노드에 keytab과 갱신용 설정을 두고 ccache를 만든다. Pod에는 keytab을 넣지 않고 ccache만 연결한다.

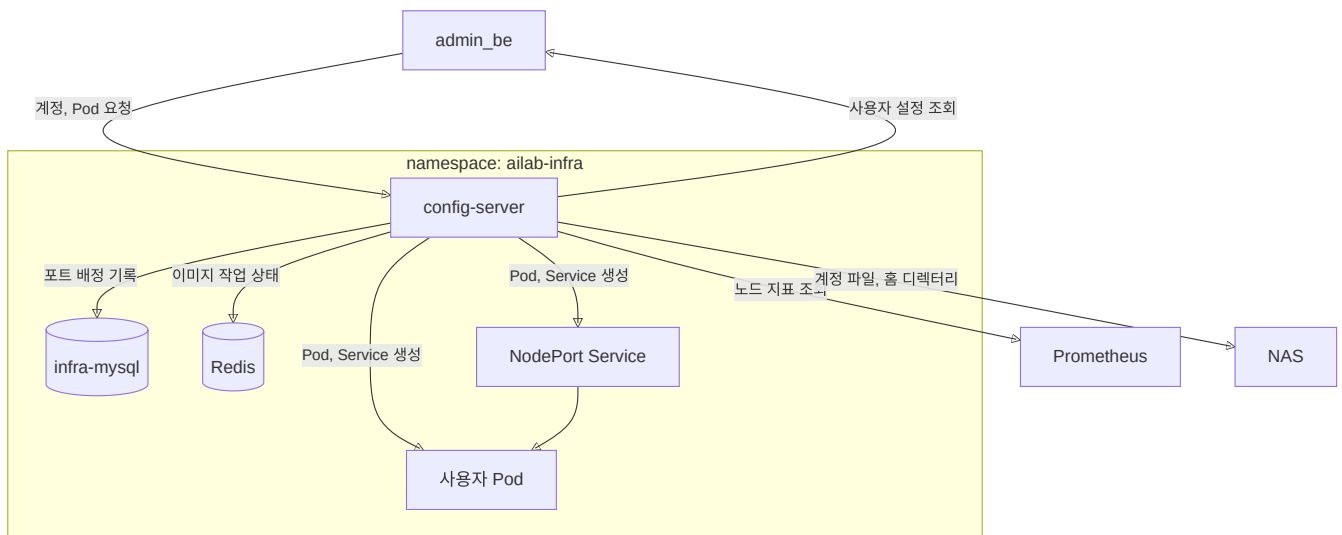
Kerberos 서버와 설정 값은 `kdc-setup`에서 확인한다.

5. 서버와 서비스가 연결되는 모습

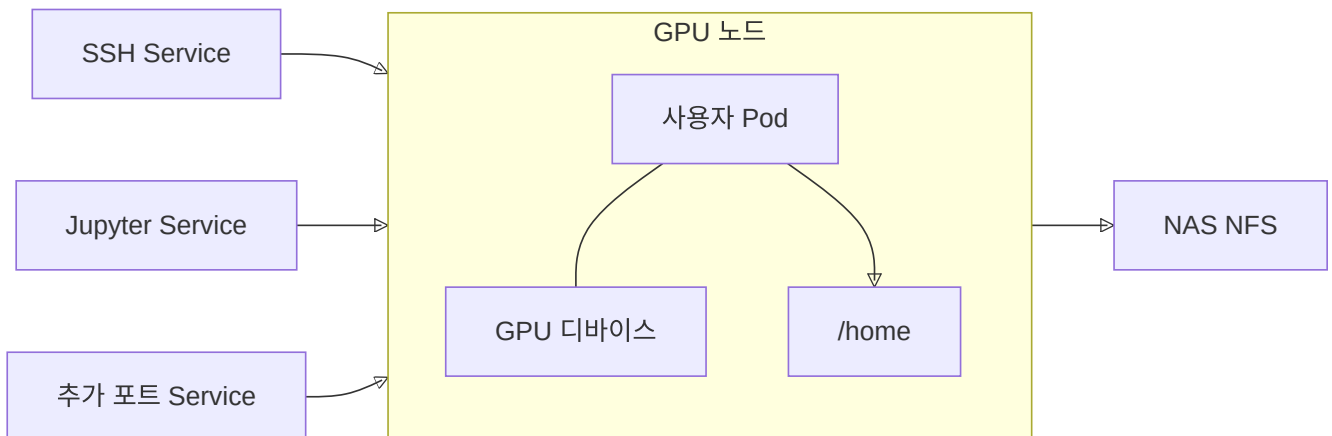
5.1 전체 연결



5.2 config-server가 연결하는 서비스



5.3 사용자 Pod



사용자 Pod는 접속용 Service, GPU 디바이스, 홈 디렉터리를 사용한다. UID와 GID, 그룹 정보는 Pod를 만들 때 넣고, 홈 디렉터리는 노드의 NFS 마운트를 통해 연결한다.

기초 개념

이 문서에서 다루는 내용

- 시스템 아키텍처에 나오는 UID/GID, 계정 파일, 권한, Kubernetes, NFS, NodePort 기본 개념
- config-server 문서를 읽을 때 자주 헷갈리는 배경 용어
- 운영 절차가 아니라 개념 설명만 먼저 보고 싶을 때 필요한 내용

빠른 이동

필요할 때	문서
실제 생성 순서를 본다	시스템 아키텍처
배포와 장애 대응을 본다	운영 매뉴얼
포트 배정 기록과 DB 구조를 본다	데이터베이스

1. 리눅스 계정과 파일 권한

1.1 UID와 GID

리눅스는 사용자와 그룹을 이름뿐 아니라 숫자로도 구분한다. 사용자 번호는 UID, 그룹 번호는 GID이다.

파일에도 사용자 이름이 아닌 UID와 GID가 소유권 정보로 기록된다. 예를 들어 파일의 소유자가 `dongmin`으로 보이더라도, 실제 디스크에는 `UID 2041`과 같은 숫자가 저장된다.

이름과 숫자의 대응 관계는 다음 파일에서 관리한다.

파일	역할
<code>passwd</code>	사용자 이름, UID, 기본 GID를 기록한다.
<code>group</code>	그룹 이름, GID, 그룹 구성원을 기록한다.

UID를 정하는 곳

NFS에 저장된 사용자 홈 디렉터리 역시 UID 숫자를 기준으로 소유권을 판단한다. 따라서 동일한 NAS를 사용하는 FARM 서버와 Kubernetes Pod는 반드시 동일한 UID 대응 관계를 사용해야 한다.

서버마다 같은 UID를 다른 사용자에게 배정하면, 특정 사용자가 다른 사용자의 홈 디렉터리를 자신의 파일처럼 읽거나 수정할 수 있다.

이를 막기 위해 config-server가 `/kube_share/passwd`를 계정 정보의 기준 파일로 사용하고 UID도 여기서 정한다.

1.2 passwd, group, shadow, sudoers

파일	저장 내용	주의사항
<code>passwd</code>	사용자 이름, UID/GID, 홈 경로, 로그인 셸	암호 정보는 없으며 누구나 읽을 수 있다.
<code>group</code>	그룹 이름, GID, 그룹 멤버	그룹 권한 관리에 사용한다.

파일	저장 내용	주의사항
shadow	사용자 암호 해시	root만 읽을 수 있다.
sudoers	sudo 사용 권한	문법 오류가 발생하면 sudo 전체가 동작하지 않을 수 있다.

일반적인 리눅스 환경에서는 `/etc/passwd`, `/etc/group` 등을 `useradd` 와 같은 명령으로 관리한다.

계정 파일을 쓰는 방법

여기서는 config-server가 NFS의 `/kube_share` 에 저장된 계정 파일을 직접 수정해 UID/GID 배정 기록으로 사용한다.

사용자 Pod에는 이 파일들을 직접 마운트하지 않는다. 대신 다음 순서로 사용자를 생성한다.

1. config-server가 `/kube_share/passwd` 에서 UID/GID를 조회한다.
2. 조회한 값을 Pod 환경변수로 주입한다.
3. 컨테이너 entrypoint가 전달받은 UID/GID로 내부 사용자를 생성한다.

계정 파일을 컨테이너의 `/etc` 에 직접 마운트하던 방식은 더 이상 쓰지 않는다.

1.3 chmod 700

`chmod` 의 세 숫자는 차례대로 다음 권한을 의미한다.

1. 파일 소유자
2. 소유 그룹
3. 기타 사용자

각 권한 값은 다음 숫자의 합으로 표현한다.

| 권한 | 값 | | ----- | -: | | 읽기 | 4 | | 쓰기 | 2 | | 실행 또는 디렉터리 진입 | 1 |

따라서 `chmod 700` 은 다음과 같은 권한 설정이다.

대상	권한
소유자	읽기, 쓰기, 진입
그룹	권한 없음
기타 사용자	권한 없음

홈 디렉터리 권한

모든 사용자 홈 디렉터리는 `chmod 700` 으로 생성한다.

현재는 사용자별 PVC를 만들지 않는다. 사용자 홈 사이의 접근은 UID 소유권과 `chmod 700` 권한으로 막는다.

2. NFS와 NAS

2.1 NFS와 NAS

NFS(Network File System)는 다른 서버의 디렉터리를 내 서버의 디렉터리처럼 쓰게 하는 방식이다.

서버가 특정 디렉터리를 외부에 공개하면(`export`), 클라이언트는 해당 경로를 자신의 파일시스템에 마운트한다. 이후 `ls`, `cat`, 파일 저장 등의 작업은 로컬 파일처럼 보이지만 실제로는 네트워크 통신을 통해 처리된다.

NAS(Network Attached Storage)는 이 NFS 저장소를 제공하는 장비다.

NAS에 저장하는 데이터

이 시스템의 NAS에는 다음 데이터가 저장된다.

- 사용자 홈 디렉터리
- `/kube_share` 계정 파일
- `infra-mysql` 데이터

따라서 NAS에 문제가 생기면 사용자 파일, 계정 관리, 데이터베이스가 함께 영향을 받는다. 이 저장소를 대신할 장비가 현재 구성되어 있지 않으므로 NAS 장애는 전체 서비스에 영향을 준다.

2.2 root_squash

`root_squash`는 NFS의 기본 보안 기능이다.

NFS 클라이언트의 `root` 사용자가 공유 디렉터리에 접근하면, NFS 서버는 그 요청을 `nobody` 같은 익명 사용자 권한으로 처리한다.

그래서 한 클라이언트 서버에서 `root` 권한을 얻더라도 NAS 파일 전체를 마음대로 바꾸기 어렵다.

홈 디렉터리를 만드는 방법

Pod 안의 프로세스가 `root`로 실행되더라도 NFS 위에서는 `root` 권한을 사용할 수 없다. 따라서 Pod가 NFS에 사용자 홈 디렉터리를 직접 생성하거나 소유권을 변경할 수 없다.

`config-server`는 Pod를 생성하기 전에 NAS에 SSH로 접속하여 다음 작업을 수행한다.

```
mkdir <사용자 홈>
chown <UID>:<GID> <사용자 홈>
chmod 700 <사용자 홈>
```

이 과정이 실패하면 Pod 생성도 실패하도록 처리한다.

2.3 NFS 파일 락

파일 잠금은 여러 프로세스가 같은 파일을 동시에 바꾸지 못하게 하는 장치다.

NFS 위의 파일 잠금은 네트워크와 별도 잠금 서비스에 의존하므로, 이 시스템에서는 계정 파일에 직접 적용하지 않는다.

현재 잠금 방식

config-server의 `LockedFile`은 NFS 파일을 수정하기 전에 config-server Pod 안의 `/tmp`에 잠금 파일을 만든다.

```
/tmp/cssh_lock*
```

현재 config-server는 한 개만 실행한다. 이 전제에서 같은 서버 안의 요청은 이 잠금으로 순서대로 처리된다.

config-server를 여러 개로 늘리면

config-server를 여러 개로 늘리면 이 방식만으로는 부족하다. 각 인스턴스의 `/tmp` 잠금 파일은 서로 보이지 않기 때문이다.

3. Kubernetes

3.1 클러스터와 노드

Kubernetes는 여러 서버를 하나의 컨테이너 실행 환경으로 묶어 관리하는 오케스트레이션 시스템이다.

- 클러스터는 여러 노드로 구성된 전체 실행 환경이다.
- 노드는 Pod가 실제로 실행되는 개별 서버이다.

Kubernetes에는 필요한 Pod와 Service의 내용을 API로 등록한다. Kubernetes는 등록된 내용과 실제 상태가 다르면 다시 맞추려고 한다.

Pod와 Service 생성

현재는 config-server가 Pod와 Service 정보를 Kubernetes API에 보내고, Kubernetes가 이를 실행한다.

3.2 Pod

Pod는 Kubernetes가 컨테이너를 실행하는 최소 단위이다.

Pod 스펙에는 다음과 같은 정보가 포함된다.

- 컨테이너 이미지
- 환경변수
- 볼륨 마운트

- GPU 개수
- 실행할 노드
- 포트 설정

사용자 Pod 규칙

여기서는 사용자 한 명마다 Pod 하나를 만든다.

config-server는 Kubernetes 클라이언트 라이브러리를 이용해 사용자 Pod 스펙을 생성한다.

보통은 Kubernetes가 실행 노드를 고르지만, 이 시스템은 config-server가 Prometheus GPU 지표로 먼저 노드를 고른 뒤 Pod에 그 노드를 지정한다.

3.3 Service와 NodePort

Pod의 IP는 Pod가 재생성될 때 변경될 수 있다. Service는 이러한 Pod에 고정된 네트워크 접근 지점을 제공한다.

NodePort 타입 Service는 모든 Kubernetes 노드의 특정 포트를 열고, 해당 포트로 들어온 요청을 대상 Pod로 전달한다.

기본 NodePort 범위는 다음과 같다.

```
30000~32767
```

사용자 Pod에 여는 포트

사용자 Pod의 각 포트마다 별도의 NodePort Service를 생성한다.

- SSH는 컨테이너 포트 22를 사용한다.
- Jupyter는 컨테이너 포트 8888을 사용한다.
- 사용자가 추가로 신청한 포트도 별도 Service로 생성한다.

config-server가 배정한 포트는 infra-mysql의 `nodeport_allocations` 테이블에 기록한다. 포트를 고를 때는 이 테이블뿐 아니라 클러스터 전체 Service의 NodePort도 함께 확인한다.

NodePort와 방화벽 매핑

Kubernetes의 NodePort는 클러스터 안에서 쓰는 포트이고, pfSense는 외부 포트를 이 NodePort로 전달한다. 외부 포트와 NodePort 번호는 같을 필요가 없다. 외부에서 접속하지 못하면 Service의 NodePort와 pfSense의 전달 대상이 같은지 확인한다.

3.4 Namespace

Namespace는 클러스터 리소스를 용도나 팀별로 구분하는 논리적 공간이다.

이 시스템의 주요 Namespace는 다음과 같다.

Namespace	용도
ailab-frontend	사용자 화면
default	admin_be
ailab-infra	config-server와 사용자 Pod
monitoring	Prometheus와 Grafana

리소스를 조회할 때는 Namespace를 정확히 지정해야 한다.

```
kubectl get pods -n ailab-infra
```

Namespace를 생략하면 다른 공간의 리소스를 조회하여 장애 원인을 놓칠 수 있다.

3.5 볼륨, hostPath, subPath

hostPath

hostPath는 노드 호스트의 특정 파일이나 디렉터리를 컨테이너에 직접 마운트하는 방식이다.

이 시스템에서는 다음 항목에 사용한다.

- 노드에 마운트된 NFS 홈 경로
- GPU 디바이스 `/dev/nvidia*`

subPath

subPath는 볼륨 전체가 아니라 볼륨 안의 특정 파일이나 디렉터리만 골라 마운트하는 옵션이다.

과거에는 `/kube_share/passwd`와 `shadow`를 컨테이너의 `/etc/passwd`, `/etc/shadow`에 직접 마운트했다. 현재 Pod 스펙에는 이 마운트가 없고, config-server가 UID/GID를 환경변수로 전달하면 컨테이너 시작 스크립트가 내부 사용자를 만든다.

코드 주석과 예전 배포 기록에는 **subPath** 방식이 남아 있을 수 있다. 현재 Pod 스펙에는 이 마운트가 없다.

3.6 imagePullPolicy와 Ready

imagePullPolicy

Pod를 생성할 때 컨테이너 이미지를 어떻게 가져올지 결정하는 정책이다.

값	동작
Always	Pod 생성 시마다 이미지를 내려받는다.

값	동작
<code>IfExists</code>	노드에 이미지가 없을 때만 내려받는다.

`Always` 는 최신 이미지를 보장하지만 시작 시간이 길어질 수 있다.

Ready

컨테이너 프로세스가 시작된 상태와 실제 요청을 받을 준비가 된 상태는 다르다.

`config-server`는 다음 순서로 Pod를 생성한다.

1. Pod 생성 요청을 보낸다.
2. Pod가 Ready가 될 때까지 상태를 확인한다.
3. Ready가 되면 NodePort Service를 만든다.
4. 생성 결과를 응답한다.

NFS 마운트가 실패하여 `FailedMount` 상태가 되면 Pod는 Ready가 되지 않으며, `config-server`의 대기는 타임아웃으로 종료된다.

3.7 RBAC와 ServiceAccount

RBAC(Role-Based Access Control)는 Kubernetes 리소스를 누가 다룰 수 있는지 정하는 권한 설정이다.

ServiceAccount는 사람이 아닌 애플리케이션이나 Pod가 사용하는 Kubernetes 내부 신원이다.

config-server의 Kubernetes 권한

`config-server`는 자신의 ServiceAccount 권한으로 Kubernetes API를 호출하여 다음 작업을 수행한다.

- Pod 생성과 삭제
- Service 생성과 삭제
- Pod 상태 조회

관련 ServiceAccount, Role, RoleBinding은 Helm 차트 배포 과정에서 함께 생성한다.

3.8 Helm

Helm은 여러 Kubernetes YAML 파일과 설정값을 한 묶음으로 관리하는 도구다.

Deployment, Service, RBAC 등의 정의를 템플릿으로 구성하고, 환경별 값만 변경해 배포할 수 있다.

`config-server`는 `Chart/` 디렉터리의 Helm 차트로 배포하며, 다음 명령이 배포 절차를 감싼다.

```
make deploy
```

4. 모니터링

4.1 Prometheus와 Pull 모델

Prometheus는 시스템 지표를 수집하고 저장하는 모니터링 시스템이다.

Prometheus는 각 서버가 데이터를 보내는 push 방식이 아니라, Prometheus가 각 대상의 HTTP 엔드포인트를 주기적으로 조회하는 **pull 방식**을 사용한다.

Prometheus 형식으로 지표를 제공하는 프로그램을 exporter라고 한다.

4.2 DCGM Exporter

DCGM은 NVIDIA GPU의 상태를 수집하는 도구이며, DCGM exporter는 GPU 지표를 Prometheus 형식으로 제공한다.

config-server는 다음 지표를 노드 선택 점수에 사용한다.

```
DCGM_FI_DEV_GPU_UTIL
DCGM_FI_DEV_FB_USED
DCGM_FI_DEV_GPU_TEMP
```

GPU 지표가 없을 때

DCGM exporter가 중단되면 해당 노드의 GPU 지표가 Prometheus에서 사라진다.

현재 쿼리는 값이 없을 때 다음 표현으로 0을 반환한다.

```
... or vector(0)
```

이 때문에 지표가 없는 노드를 사용률이 0인 노드, 즉 가장 한가한 노드로 잘못 판단할 수 있다.

지표 없음과 실제 사용률 0은 구분해야 한다.

4.3 PromQL 기본 문법

PromQL은 Prometheus에 저장된 지표를 조회하는 질의 언어이다.

예를 들어 farm5의 GPU 평균 사용률은 다음과 같이 조회한다.

```
avg(DCGM_FI_DEV_GPU_UTIL{Hostname="farm5"})
```

주요 문법은 다음과 같다.

표현	의미
<code>metric{label="value"}</code>	특정 레이블을 가진 지표를 조회한다.
<code>avg(...)</code>	여러 GPU 값의 평균을 계산한다.
<code>or vector(0)</code>	조회 결과가 없을 경우 0을 반환한다.

`or vector(0)` 은 계산을 단순화하지만, 장애로 인한 지표 부재까지 정상적인 0으로 처리한다는 문제가 있다.

4.4 Grafana

Grafana는 Prometheus에 저장된 지표를 대시보드 형태로 보여주는 시각화 도구이다.

관리자는 다음 주소에서 노드와 GPU 상태를 확인할 수 있다.

```
http://210.94.179.18:30080
```

PromQL을 직접 실행하려면 Prometheus에 port-forward로 접속한다.

5. 네트워크와 방화벽

5.1 공인 IP, 사설 IP, NAT

- **사설 IP**는 내부 네트워크에서만 사용하는 주소이다.
- **공인 IP**는 인터넷에서 직접 접근할 수 있는 주소이다.
- **NAT**는 사설 IP와 공인 IP 사이에서 주소나 포트를 변환하는 기술이다.
- **포트포워딩**은 특정 공인 IP와 포트로 들어온 요청을 내부 서버의 특정 포트로 전달하는 규칙이다.

예시는 다음과 같다.

```
공인 IP:9300 → farm6:30000
```

5.2 pfSense

pfSense는 외부 네트워크와 내부 서버 사이에서 방화벽과 라우터 역할을 한다.

공인 IP	정책
210.94.179.18	사전에 허용한 포트만 통과시킨다.
210.94.179.19	외부 포트를 사용자 Pod의 NodePort로 전달한다.

외부 접속이 안 될 때

NodePort 번호만 보고 외부 접속 주소를 추정하지 않는다. Service의 NodePort, pfSense가 전달하는 대상, 사용자에게 안내한 외부 주소를 함께 확인한다.

6. 저장소와 동시성

6.1 트랜잭션과 SELECT ... FOR UPDATE

트랜잭션은 여러 SQL 작업을 하나의 작업 단위로 묶는 기능이다.

트랜잭션 내부의 작업은 모두 성공하거나 모두 취소된다.

`SELECT ... FOR UPDATE` 는 조회한 행에 잠금을 걸어, 현재 트랜잭션이 끝날 때까지 다른 트랜잭션이 해당 행을 수정하지 못하게 한다.

NodePort를 지정하는 순서

NodePort 배정은 다음 순서로 처리한다.

1. 트랜잭션을 시작한다.
2. 사용 중인 포트 목록을 `FOR UPDATE` 로 조회한다.
3. 빈 포트를 선택한다.
4. 배정 정보를 insert한다.
5. 트랜잭션을 commit한다.

두 개의 Pod 생성 요청이 동시에 들어오더라도 같은 포트가 중복 배정되는 것을 방지한다.

6.2 Redis

Redis는 메모리 기반 키-값 저장소이다.

이 시스템에서는 사용자 이미지 저장과 로드 작업의 진행 상태만 기록한다.

```
img:<username>
```

이미지 commit 또는 save 작업이 오래 걸리므로, Redis는 현재 작업 단계를 API에서 조회할 수 있도록 상태판 역할을 한다.

6.3 정보별 기준 저장 위치

같은 정보가 여러 곳에 있으면 값이 달라질 수 있다. 그래서 정보마다 어느 곳의 값을 기준으로 사용할지 정해 둔다.

이 시스템에서는 다음 위치의 값을 기준으로 사용한다.

정보	기준으로 보는 곳
사용자 UID/GID	/kube_share/passwd
요청 상태	admin_be DB의 requests 테이블
NodePort 배정	infra-mysql의 nodeport_allocations 테이블

admin_be가 config-server에 UID를 전달하더라도 config-server는 전달값을 신뢰하지 않고 /kube_share/passwd 를 기준으로 동작한다.

반대로 요청의 PENDING, FULFILLED, DELETED 상태는 admin_be가 관리하며 config-server는 해당 상태를 저장하지 않는다.

6.4 삭제 표시(soft delete)

삭제 표시는 데이터베이스 행을 지우지 않고 삭제되었다는 상태만 기록하는 방식이다.

admin_be의 요청 상태 DELETED 가 이에 해당한다.

주요 장점은 다음과 같다.

- 삭제 이력을 보존한다.
- 운영 감사와 장애 추적이 가능하다.
- 필요할 경우 데이터를 복구할 수 있다.

7. 실행 환경

7.1 Flask와 Gunicorn

Flask는 config-server에 사용한 Python 웹 프레임워크이다.

Flask의 내장 서버는 개발용이므로 운영 환경에서는 Gunicorn을 사용한다. Gunicorn은 여러 워커 프로세스로 Flask 애플리케이션을 실행하여 동시 요청을 처리한다.

config-server의 로컬 파일 락은 Gunicorn의 여러 워커가 동시에 계정 파일을 수정하는 상황을 제어하는 역할도 한다.

7.2 WAS와 Spring Boot

WAS(Web Application Server)는 웹 요청을 받아 비즈니스 로직을 실행하는 서버를 의미한다.

이 시스템에서는 Spring Boot로 작성한 `admin_be`가 WAS 역할을 수행한다.

운영 문서나 대화에서 “WAS”라고 표현하면 일반적으로 `admin_be`를 의미한다.

7.3 Swagger

Swagger는 API 목록과 요청 형식을 웹 페이지로 제공하고, 브라우저에서 직접 API를 호출할 수 있게 하는 도구이다.

config-server Swagger 주소는 다음과 같다.

```
http://210.94.179.18:30082/apidocs/
```

Swagger를 사용할 때

config-server API에는 별도의 인증 기능이 없다.

따라서 Swagger의 `Try it out` 기능은 단순한 테스트가 아니라 실제 Kubernetes와 NAS 리소스를 생성·수정·삭제할 수 있는 운영 명령이다.

Swagger 호출은 실제 인프라를 변경하는 작업이므로 운영 권한과 동일하게 취급해야 한다.

핵심 구조 요약

여기서는 다음 방식으로 동작한다.

1. 사용자 식별과 파일 권한의 기준은 UID/GID이다.
2. UID/GID는 `/kube_share/passwd`에 적힌 값을 사용한다.
3. 사용자 홈은 NFS에 저장하며 `chmod 700`으로 다른 사용자의 접근을 막는다.
4. `root_squash` 때문에 사용자 홈은 Pod 생성 전에 NAS에서 미리 생성해야 한다.
5. 사용자 한 명당 하나의 Pod와 여러 NodePort Service를 생성한다.
6. 실행 노드는 config-server가 Prometheus GPU 지표를 바탕으로 선택한다.
7. NodePort는 infra-mysql에서 트랜잭션과 행 잠금으로 배정한다.
8. 외부 접속은 pfSense가 외부 포트를 해당 NodePort로 전달해야 가능하다.

9. NAS, 계정 파일, NodePort DB 등 주요 기준 정보가 손상되면 전체 시스템에 영향을 줄 수 있다.
10. Swagger와 config-server API는 인증이 없으므로 실제 인프라 관리 인터페이스로 취급해야 한다.

데이터베이스

이 문서에서 다루는 내용

- infra-mysql이 어떤 테이블을 가지고 있고 config-server가 어디에 쓰는지
- NodePort를 동시에 안전하게 배정하기 위해 어떤 기록과 잠금을 쓰는지
- DB를 확인하거나 복구할 때 Kubernetes 상태와 함께 무엇을 봐야 하는지

빠른 이동

필요할 때	문서
승인부터 Pod 생성까지 전체 흐름을 본다	시스템 아키텍처
운영 중 포트 충돌이나 잔여 기록을 확인한다	운영 매뉴얼
API 응답과 오류 형식을 확인한다	API 레퍼런스

1. 사용하는 데이터베이스

DB	관리 주체	용도	관련 저장소
infra-mysql pod_port_db	config-server 전용	NodePort 배정 기록과 FARM 노드 Kerberos 정리 재 시도 목록을 저장한다.	이 저장소(admin_infra)
admin_be MySQL	admin_be(W AS)	신청·사용자 정보를 관리한다.	별도 저장소이며, 이 문서에서는 이름만 언급한다.

NodePort는 동시에 두 요청이 들어와도 같은 번호를 두 번 배정하면 안 된다. config-server는 포트를 배정할 때 infra-mysql의 행 잠금을 사용해 이 문제를 막는다.

신청과 사용자 정보는 admin_be MySQL에서 관리한다. 두 시스템은 서로의 DB를 직접 조회하지 않고 API로만 통신한다.

포트 배정 기록은 infra-mysql에 남는다. 다만 실제 Service가 남아 있는지도 Kubernetes에서 함께 확인해야 한다.

2. infra-mysql 배포 형태

infra-mysql은 `infra-sql/` 디렉터리의 매니페스트로 배포한다.

리소스	내용
StatefulSet <code>infra-mysql</code>	<code>mysql:8.0.36</code> 이미지, replicas는 1이고 Namespace는 <code>ailab-infra</code> 이다.
Service <code>infra-mysql</code>	ClusterIP 타입이며 3306 포트를 제공. config-server는 <code>db.host: "infra-mysql"</code> 로 접속한다.
StorageClass <code>sc-mysql</code>	NFS CSI인 <code>nfs.csi.k8s.io</code> 를 사용. NAS export를 백엔드로 사용하며 <code>reclaimPolicy: Retain</code> , <code>nfsvers=3</code> 으로 설정.
PVC <code>mysql-data</code>	StatefulSet의 <code>volumeClaimTemplate</code> 으로 생성되며 용량은 10Gi. <code>/var/lib/mysql</code> 에 마운트.

StatefulSet은 Pod 이름과 스토리지를 고정하는 Kubernetes 워크로드이다. 따라서 infra-mysql Pod의 이름은 항상 `infra-mysql-0` 이다.

계정과 DB 생성 주체

계정과 데이터베이스는 MySQL 공식 이미지의 초기화 동작이 생성한다.

공식 MySQL 이미지는 데이터 디렉터리가 비어 있는 최초 기동 시 다음 환경변수를 읽어 초기화를 수행한다.

환경변수	역할
<code>MYSQL_DATABASE</code>	지정한 데이터베이스를 생성. 이 시스템에서는 <code>pod_port_db</code> .
<code>MYSQL_USER</code>	일반 사용자 계정을 만든다. 이 시스템에서는 <code>pod_port_user</code> 다.
<code>MYSQL_PASSWORD</code>	생성한 사용자 계정의 비밀번호를 설정.

이미지는 생성한 사용자에게 `MYSQL_DATABASE` 로 지정한 데이터베이스의 전체 권한을 부여한다.

즉 `CREATE USER`, `GRANT` 같은 계정 생성과 권한 부여는 MySQL 이미지가 처음 시작할 때 처리한다.

현재 상태는 다음과 같다.

- 배포 매니페스트와 코드에 수동 `GRANT` 구문이 없다.
- 계정과 권한 생성은 MySQL 이미지의 초기화 동작에 위임한다.
- 별도의 init SQL 파일이나 초기 스키마 스크립트가 없다.

config-server는 Kubernetes Secret인 `config-server-db-secret` 에서 비밀번호를 `DB_PASSWORD` 환경변수로 주입받는다. infra-mysql 측 비밀번호는 배포 manifest나 Secret을 `kubectrl` 로 조회한다.

3. 스키마와 DDL 관리

infra-mysql에서 현재 코드가 사용하는 테이블은 두 개다.

테이블	쓰는 곳	용도
<code>nodeport_allocations</code>	<code>main.py</code>	사용자 Pod의 NodePort 배정 기록
<code>krb5_cleanup_pending</code>	<code>main.py</code> , <code>reconcile_krb5.py</code>	FARM 노드에서 실패한 Kerberos 파일 정리를 다음 CronJob 실행 때 다시 시도할 목록

`server_nodes` 와 같은 다른 테이블은 현재 코드에서 사용하지 않는다.

3.1 `nodeport_allocations`

컬럼	역할
<code>username</code>	포트를 소유한 사용자를 기록한다.
<code>pod_name</code>	포트를 사용하는 사용자 Pod 이름이다. 포트 해제와 동기화 작업의 키로 사용한다.
<code>node_name</code>	Pod가 배치된 Kubernetes 노드를 기록한다.
<code>internal_port</code>	컨테이너 내부 포트이다. 예시는 22, 8888 등이다.
<code>node_port</code>	외부 접근을 위해 배정한 NodePort이다. 기본 Kubernetes 범위는 30000~32767이다.
<code>purpose</code>	포트의 용도를 기록한다. 예시는 <code>ssh</code> , <code>jupyter</code> , <code>custom</code> 등이다.

3.2 `krb5_cleanup_pending`

Kerberos를 쓰는 배포에서는 Pod 또는 계정을 지울 때 FARM 노드의 keytab, timer, ccache도 지운다. 이 작업이 실패하면 config-server는 아래 정보를 `krb5_cleanup_pending` 에 기록한다. 30분마다 실행되는 `reconcile_krb5.py` 는 이 기록을 다시 처리하고, 성공하면 행을 지운다.

컬럼	역할
<code>username</code>	정리할 사용자 이름
<code>node_name</code>	정리 작업이 실패한 FARM 노드
<code>failed_at</code>	마지막으로 정리에 실패한 시각

코드의 `ON DUPLICATE KEY UPDATE` 구문상 `username`, `node_name` 조합에는 중복을 막는 키가 있어야 한다. 정확한 기본 키·인덱스·컬럼 타입은 코드만으로 확인할 수 없으므로, 재구축할 때는 기존 DB의 DDL을 사용한다.

3.3 DDL 생성 방식

DDL(Data Definition Language)은 테이블 구조를 생성하거나 변경하는 SQL이다. 대표적인 구문은 `CREATE TABLE`, `ALTER TABLE` 이다.

현재 코드에는 `CREATE TABLE` 구문이 없다.

따라서 DB를 새로 구축하면 `nodeport_allocations` 와 `krb5_cleanup_pending` 을 직접 만들어야 한다. 자동으로 테이블을 만드는 기능은 없다.

위 컬럼 목록은 코드의 `INSERT` 와 `SELECT` 에서 실제로 쓰는 컬럼을 기준으로 작성했다.

자동 증가 ID, 기본 키, 인덱스, 컬럼 타입은 코드만으로 확인할 수 없다. DB를 다시 만들 때는 기존 DB에서 아래 결과를 먼저 가져와 사용한다.

```
SHOW CREATE TABLE nodeport_allocations;
SHOW CREATE TABLE krb5_cleanup_pending;
```

기존 스키마 백업이 없을 때만 위 컬럼 목록을 바탕으로 새 DDL을 작성한다.

재구축용 DDL 실행 절차

```
kubectl exec -it infra-mysql-0 -n ailab-infra -- \
mysql -u pod_port_user -p pod_port_db
```

MySQL 접속 후 두 테이블의 생성 DDL을 실행한다.

```
CREATE TABLE nodeport_allocations (...);
CREATE TABLE krb5_cleanup_pending (...);
```

실제 운영 환경에서는 두 테이블의 `SHOW CREATE TABLE` 결과를 사용해야 한다.

4. Flask에서의 사용 패턴

4.1 접속 방식

config-server의 MySQL 라이브러리는 PyMySQL이다.

`utils.py` 의 `get_db_connection()` 함수가 다음 환경변수를 읽어 DB에 접속한다.

환경변수	역할
<code>DB_HOST</code>	MySQL 호스트이다.
<code>DB_USER</code>	MySQL 사용자이다.

환경변수	역할
DB_PASSWORD	MySQL 비밀번호이다.
DB_NAME	사용할 데이터베이스 이름이다.

포트 배정, 해제, 동기화와 같은 작업 단위마다 새로운 DB 연결을 생성하고, 작업이 끝나면 `finally` 구문에서 연결을 닫는다.

`autocommit=False` 로 동작하므로 SQL 실행 후 명시적으로 `commit()` 을 호출해야 변경 사항이 반영된다.

4.2 포트 배정 트랜잭션

NodePort 배정 트랜잭션은 `main.py` 의 `allocate_nodeports()` 함수가 담당한다.

처리 순서는 다음과 같다.

1. 사용 중인 포트 목록을 `FOR UPDATE` 로 조회한다.

```
SELECT node_port
FROM nodeport_allocations
FOR UPDATE;
```

1. 클러스터 모든 namespace의 Service가 쓰는 NodePort도 조회해 사용 중인 목록에 합친다. 이 조회가 실패하면 오류를 로그에 남기고 MySQL 목록만으로 계속 진행한다. 이때 다른 Service가 이미 쓰는 포트를 고르면 Service 생성 단계에서 실패할 수 있다.
2. NodePort 범위인 30000~32767에서 사용하지 않는 포트를 찾는다.
3. 요청한 포트 개수만큼 `nodeport_allocations` 에 행을 삽입한다.
4. 모든 삽입이 성공하면 `conn.commit()` 으로 트랜잭션을 확정한다.
5. 예외가 발생하면 `conn.rollback()` 으로 전체 작업을 취소한다.

트랜잭션은 여러 SQL 작업을 묶어서 모두 반영하거나, 하나라도 실패하면 모두 취소하는 기능이다.

4.3 SELECT ... FOR UPDATE를 사용하는 이유

두 개의 승인 요청이 거의 동시에 들어오면 두 트랜잭션이 같은 시점에 동일한 빈 포트 목록을 읽을 수 있다.

잠금이 없다면 다음과 같은 경쟁 상태가 발생할 수 있다.

1. 트랜잭션 A가 30042번 포트가 비어 있다고 판단한다.
2. 트랜잭션 B도 30042번 포트가 비어 있다고 판단한다.
3. A와 B가 모두 30042번 포트를 배정하려고 한다.

4. 같은 NodePort가 두 사용자에게 중복 배정된다.

`SELECT ... FOR UPDATE` 는 현재 작업이 끝날 때까지 조회한 행을 다른 작업이 바꾸지 못하게 한다.

두 번째 요청은 첫 번째 요청이 `commit` 하거나 `rollback` 할 때까지 기다린다. 그래서 동시에 들어온 포트 배정 요청도 차례대로 처리된다.

이 구조를 통해 NodePort 중복 배정을 방지한다.

4.4 포트 해제와 동기화

함수	동작
<code>release_nodeports(pod_name)</code>	해당 <code>pod_name</code> 과 연결된 행을 삭제한 후 <code>commit</code> 한다.
<code>reconcile_nodeport_allocations(namespace)</code>	MySQL 기록과 Kubernetes Service를 비교해 남은 기록을 정리한다.

`release_nodeports`

`release_nodeports(pod_name)` 은 특정 Pod가 사용하던 NodePort 배정 정보를 삭제한다.

처리 방식은 다음과 같다.

```
DELETE FROM nodeport_allocations
WHERE pod_name = <pod_name>;
```

삭제 후 `commit()` 을 호출한다.

`reconcile_nodeport_allocations`

`reconcile_nodeport_allocations(namespace)` 는 infra-mysql 기록과 Kubernetes의 실제 NodePort Service 목록을 비교한다.

이 동기화 작업은 Kubernetes에 실제로 남아 있는 Service를 기준으로 한다.

`app=ai lab-nodeport` 라벨과 같은 `pod_name` 을 가진 Service가 Kubernetes에 하나도 없는데 infra-mysql에 그 Pod의 배정 기록이 남아 있으면 해당 Pod의 행을 삭제한다. 포트별 Service를 하나씩 비교하는 방식은 아니다. 따라서 같은 Pod의 Service가 하나라도 남아 있으면 그 Pod의 DB 행은 모두 남는다.

포트 배정 직전에 이 함수를 호출하지만, Kubernetes API와 DB를 매 요청마다 전체 조회하지 않도록 마지막 시도부터 5분(300초)이 지나야 다시 실행한다. 실패한 시도도 이 5분 간격에 포함한다.

동기화에 실패하면 트랜잭션을 `rollback`한다. 다만 동기화 실패는 치명적 오류로 처리하지 않으며, 포트 배정 작업은 계속 진행한다.

5. 운영 절차

5.1 상태 확인

접속 방법은 [운영 매뉴얼](#)의 관리 주소와 명령 표를 따른다.

`kubectl exec` 로 `infra-mysql-0` Pod 내부에서 MySQL 클라이언트를 실행한다. 비밀번호는 Kubernetes Secret에서 조회한다.

```
kubectl exec -it infra-mysql-0 -n ailab-infra -- \
mysql -u pod_port_user -p pod_port_db
```

현재 점유 중인 포트 전체는 다음 쿼리로 조회한다.

```
SELECT
    username,
    pod_name,
    node_port,
    purpose
FROM nodeport_allocations
ORDER BY node_port;
```

특정 사용자의 포트 배정 현황은 다음 쿼리로 조회한다.

```
SELECT *
FROM nodeport_allocations
WHERE username = '<username>';
```

FARM 노드에서 다시 정리해야 할 Kerberos 파일 목록은 다음 쿼리로 확인한다.

```
SELECT username, node_name, failed_at
FROM krb5_cleanup_pending
ORDER BY failed_at;
```

5.2 백업

현재 자동 백업은 구성되어 있지 않다.

배포 매니페스트와 코드 어디에도 다음 구성은 존재하지 않는다.

- `mysqldump` 자동 실행

- Kubernetes CronJob
- 외부 백업 스크립트
- 정기 스냅샷 설정

현재 데이터 보호 장치는 StorageClass의 다음 설정뿐이다.

```
reclaimPolicy: Retain
```

Retain 정책은 PVC나 StatefulSet을 삭제하더라도 NFS 위의 PV 데이터를 자동으로 제거하지 않도록 한다.

다만 이는 백업이 아니라 삭제 방지 장치이다. 파일 손상, 잘못된 SQL 실행, NAS 장애에는 대응하지 못한다.

infra-mysql에는 NodePort 배정 기록과 Kerberos 정리 재시도 기록이 있다. 데이터가 사라지면 Kubernetes의 실제 Service를 보고 포트 기록을 다시 만들 수 있다. Kerberos 정리 재시도 기록은 삭제 실패 로그와 FARM 노드의 파일을 대조해 다시 만들어야 한다.

스키마 정의는 코드에서 자동으로 생성되지 않으므로 두 테이블의 DDL을 함께 백업해야 한다.

```
SHOW CREATE TABLE nodeport_allocations;
SHOW CREATE TABLE krb5_cleanup_pending;
```

5.3 재구축 순서

infra-mysql 재구축은 다음 순서로 진행한다.

1. **nfs-mysql.yaml** 을 적용하여 StorageClass **sc-mysql** 을 생성한다.
2. **infra-mysql.yaml** 을 적용하여 StatefulSet과 Service를 생성한다.
3. MySQL 최초 기동 과정에서 **pod_port_db** 데이터베이스와 **pod_port_user** 계정이 자동 생성되는지 확인한다.
4. **nodeport_allocations** , **krb5_cleanup_pending** 테이블의 DDL을 수동으로 실행한다.
5. config-server를 재시작한다.
6. 사용자 Pod 생성 요청을 한 건 실행한다.
7. infra-mysql에 NodePort 배정 행이 정상적으로 삽입되는지 확인한다.
8. Kubernetes에 NodePort Service가 정상적으로 생성되는지 확인한다.

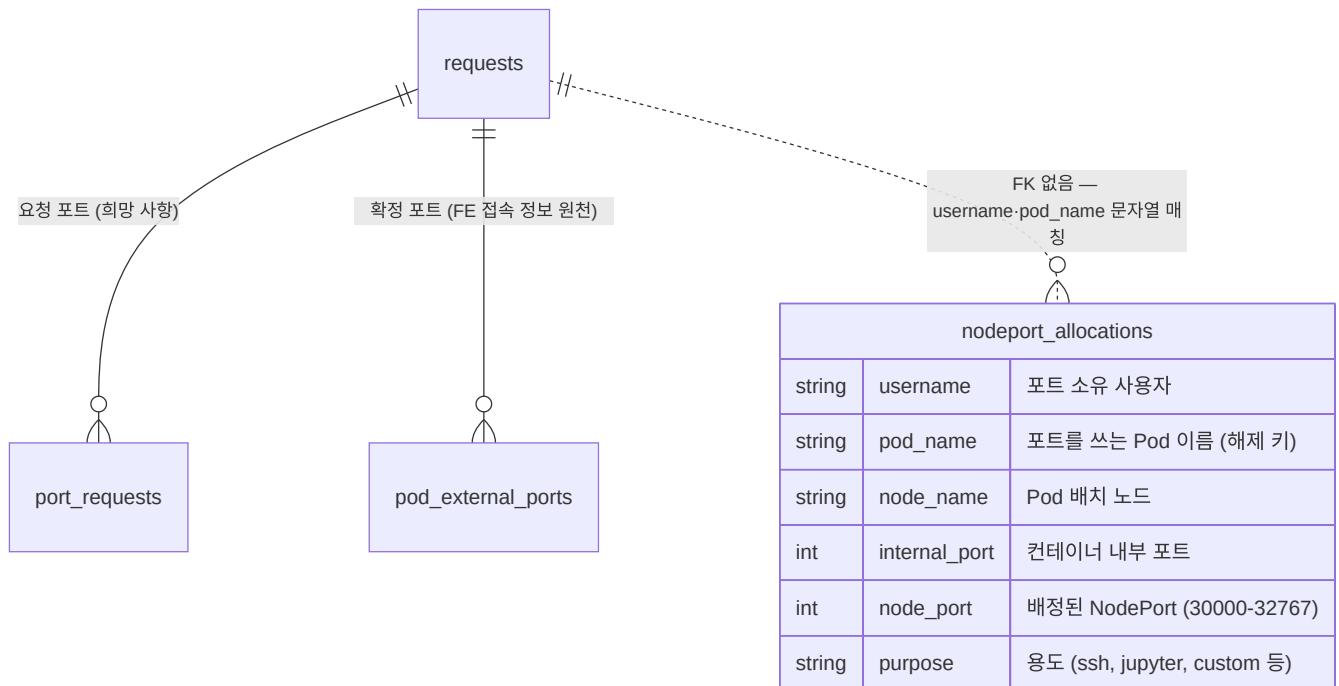
자동 스키마 bootstrap이 없으므로 4단계의 수동 DDL 실행은 필수이다.

6. admin_be DB와 포트 정보를 비교하는 방법

다음 ERD는 포트 요청과 배정 정보를 두 데이터베이스에서 어떻게 비교하는지 보여 준다.

`nodeport_allocations`의 컬럼은 코드에서 실제 사용하는 컬럼을 기준으로 작성한다. 타입과 기본 키 구성은 미확인 상태이므로 개념 수준으로 표기한다.

admin_be의 테이블은 별도 저장소에서 관리하므로 테이블 이름과 관계만 표현하고 상세 컬럼은 생략한다.



infra-mysql과 admin_be MySQL은 물리적으로 분리된 데이터베이스이다.

두 데이터베이스 사이에는 외래 키가 존재하지 않는다.

`nodeport_allocations`와 admin_be 테이블은 `username`, `pod_name` 문자열로만 비교할 수 있다. 외래 키가 없으므로 두 DB를 SQL 조인으로 조회할 수 없다.

대조 작업은 다음 방식으로 수행한다.

1. admin_be MySQL에서 요청과 확정 포트 정보를 조회한다.
2. infra-mysql에서 `nodeport_allocations`를 조회한다.
3. `username`과 `pod_name`을 기준으로 결과를 비교한다.
4. 불일치가 있으면 Kubernetes의 실제 Service 상태까지 함께 확인한다.

ERD의 점선은 외래 키 없이 `username`, `pod_name`을 비교한다는 표시다.

핵심 구조 요약

이 시스템의 데이터베이스 구조는 다음과 같다.

1. MySQL은 infra-mysql과 admin_be MySQL 두 개로 분리된다.
2. infra-mysql은 NodePort 배정 기록과 Kerberos 정리 재시도 기록을 관리한다.
3. admin_be MySQL은 사용자와 신청 정보를 관리한다.

4. 두 DB는 직접 연결하지 않고 API를 통해 통신한다.
5. infra-mysql의 데이터베이스와 계정은 MySQL 공식 이미지가 최초 기동 시 생성한다.
6. `nodeport_allocations`, `krb5_cleanup_pending` 테이블은 자동으로 생성되지 않으므로 수동 DDL이 필요하다.
7. NodePort 배정은 `SELECT ... FOR UPDATE` 와 트랜잭션으로 처리한다.
8. 포트 해제는 `pod_name` 을 기준으로 DB 행을 삭제한다.
9. Kubernetes 실제 Service와 DB 기록의 차이는 reconcile 작업으로 정리한다.
10. 자동 백업은 없으며 `reclaimPolicy: Retain` 은 삭제 방지 장치일 뿐 백업은 아니다.
11. 두 데이터베이스 사이에는 외래 키가 없으며 `username` 과 `pod_name` 으로 연결해 비교한다.
12. 장애 조사 시 infra-mysql, admin_be MySQL, Kubernetes Service 상태를 각각 조회해 비교한다.

처음 작업할 때

이 문서에서 다루는 내용

- `develop`, `main`, `feature/*`, `hotfix/*` 브랜치의 역할
- 로컬에서 config-server 이미지를 띄워 Swagger와 `/health` 를 확인하는 방법
- 클러스터에서 배포 결과를 확인할 위치와 PR 규칙

빠른 이동

필요할 때	문서
운영 배포와 장애 대응을 바로 본다	운영 매뉴얼
Helm 값과 차트 구조를 본다	Helm 차트 레퍼런스
config-server가 실제로 무슨 일을 하는지 먼저 본다	개요

1. 브랜치와 배포

코드는 `develop` 에서 모으고, `main` 에 병합되면 운영 서버에 배포된다.

브랜치	역할	배포 여부	비고
<code>main</code>	운영 서버에 배포되는 브랜치	O	PR을 병합하면 배포한다.
<code>develop</code>	기능을 합쳐 테스트하는 브랜치	X	기능 개발이 끝난 뒤 이 브랜치로 PR을 만든다.
<code>feature/*</code>	기능별 작업 브랜치	X	<code>develop</code> 에서 만든다.
<code>hotfix/*</code>	긴급 수정 브랜치	X	<code>main</code> 에서 만들고, 배포하려면 다시 <code>main</code> 에 병합한다.

2. 로컬에서 API 확인하기

config-server는 Python 3.10 slim과 gunicorn으로 만든 컨테이너로 실행한다. 로컬에서는 Docker로 띄워 Swagger와 `/health` 응답을 확인할 수 있다.

```
cd admin_infra/config-server

# 이미지 빌드
docker build -t config-server:local .

# 실행 (컨테이너 포트 8000)
docker run --rm -p 8000:8000 config-server:local
```

- Swagger: `http://localhost:8000/apidocs/`
- 상태 확인: `http://localhost:8000/health`

계정 생성과 Pod 생성은 Kubernetes API, infra-mysql, `/kube_share` NFS, NAS SSH가 있어야 동작한다. 로컬 컨테이너에서는 Swagger와 코드 변경을 확인하고, 실제 생성은 클러스터에서 시험 사용자 한 명으로 확인한다.

3. 클러스터에서 확인할 것

- 운영 config-server는 FARM 클러스터의 `ailab-infra` namespace에서 실행한다.
- `kubectl` 은 control-plane에서 실행한다. CI/CD는 운영 서버 `farm8` 에 SSH로 접속해 `helm upgrade` 를 실행한다.
- 배포 뒤 확인할 명령은 운영 매뉴얼에 있다.

```
# 배포된 config-server 확인
kubectl get pods -n ailab-infra | grep config-server
```

4. 커밋과 PR

1. `develop` 에서 `feature/#이슈번호-작업명` 브랜치를 만든다. 예: `feature/#155-scheduler`
2. 커밋 메시지는 `[분류] #이슈번호 설명` 으로 쓴다. 예: `[feat] #4 메인 기능 만들기`
3. 작업이 끝나면 `feature` 에서 `develop` 으로 PR을 만든다.
4. 정기 배포는 `develop` 에서 `main` 으로 PR을 만든다. 제목은 `[deploy] develop -> main (설명)` 으로 쓴다.
5. 한 명 이상 승인한 뒤 병합한다. `main` 에 병합하면 운영 서버에 배포된다.

배포 전에 다른 사람이 같은 클러스터에서 테스트하거나 Helm 값을 바꾸고 있는지 확인한다. 현재 실행 중인 이미지가 어느 커밋에서 만들어졌는지도 확인한 뒤 배포한다.

운영 매뉴얼

이 문서에서 다루는 내용

- config-server 배포 상태 확인, 운영 주소, 주요 리소스 이름
- 계정과 Pod 생성·삭제 뒤에 무엇을 점검해야 하는지
- 오류 응답, 잔여 리소스, NodePort 기록, Secret을 어떤 순서로 확인해야 하는지

빠른 이동

필요할 때	문서
생성 순서와 실패 지점을 먼저 본다	시스템 아키텍처
API 입력과 오류 형식을 본다	API 레퍼런스
포트 DB와 복구 기준을 본다	데이터베이스
Helm 값과 Secret 이름을 본다	Helm 차트 레퍼런스

config-server 배포, 계정과 Pod 작업, 문제 확인·복구 방법을 정리한다. 계정·Pod·포트가 만들어지는 순서는 [시스템 아키텍처](#), 용어는 [기초 개념](#)에서 확인한다. Helm 값과 Secret 이름은 [Helm 차트 레퍼런스](#)에 있다. **keytab·개인키·비밀번호의 실제 값은 문서나 로그에 적지 않는다.**

1. 작업 전에 확인할 것

작업을 시작하기 전에 다음을 지킨다.

1. **계정과 Pod는 따로 지운다.** 계정을 지운 뒤 Pod가 남거나, Pod를 지운 뒤 계정과 홈이 남을 수 있다. 삭제할 때는 두 API를 모두 호출했는지 확인한다.
2. **작업 전후 값을 기록한다.** 사용자 이름, Pod 이름, UID/GID, 선택 노드, NodePort를 적어 둔다. 문제가 생기면 이 기록으로 어느 단계에서 멈췄는지 확인한다.
3. **API 오류 뒤에는 같은 요청을 바로 반복하지 않는다.** 계정·Pod 생성은 여러 외부 시스템을 차례로 바꾼다. 오류 응답의 `stage` 와 `rollback` 을 먼저 보고 일부만 만들어진 것이 없는지 확인한다.
4. **비밀 값은 출력하지 않는다.** keytab, SSH 개인키, DB 비밀번호, shadow 내용은 화면·로그·티켓 어디에도 남기지 않는다.

- 5. **Kubernetes 리소스를 직접 지우기 전에 기록을 확인한다.** config-server와 MySQL에 남은 포트 배정 기록을 먼저 본다. Pod나 Service 한쪽만 지우면 기록과 실제 상태가 더 달라질 수 있다.
- 6. **예전 chart·스크립트는 복구에 쓰지 않는다.** 레포에 남아 있는 레거시(ContainerSSH, 사용자 PVC chart, restart_*.sh)는 현행 구조와 무관하다(10절 금지 사항).
- 7. **config-server 제어 API는 승인된 경로에서만 접근한다.** 이 API에는 인증이 없으므로(API 레퍼런스의 공통 사항), WAS와 승인된 운영망 밖으로 노출되지 않게 유지한다.
- 8. **배포 뒤에도 실제 동작을 확인한다.** 배포가 끝나면 승인 한 건으로 계정·Pod·접속 포트가 만들어지는지 확인한다.

2. 이름과 접속 주소

2.1 주요 이름과 포트

항목	값
운영 네임스페이스	ailab-infra
Helm release	containersssh-config-server
config-server Deployment	containersssh-config-server
config-server Service	containersssh-config-service
config-server HTTP 포트	Service 80 → 컨테이너 8000
config-server 외부 NodePort	30082
사용자 Pod 접두사	ailab-
사용자 NodePort 범위	30000-32767
Pod Ready 최대 대기	300초 (배포 브랜치에 따라 다를 수 있음)
Pod 삭제 최대 대기	60초
진행 상태 보존	최종 갱신 후 1시간

config-server 관련 리소스에는 과거 이름인 containersssh 접두사가 남아 있다. 현재 ContainerSSH 프록시는 배포하지 않는다.

2.2 서비스별 namespace

namespace는 Kubernetes 리소스를 나누어 두는 단위다.

namespace	배포 대상	분리한 이유
ailab-infra	config-server, infra-mysql, redis, 사용자 Pod 전체	사용자 Pod를 만드는 서비스와 Pod를 함께 둔다.

namespace	배포 대상	분리한 이유
ailab-frontend	admin_fe (정적 서빙)	화면 파일을 제공하는 서비스만 둔다.
default (admin-prod)	admin_be (Spring Boot WAS)	신청과 승인 정보를 관리한다. config-server가 사용자 설정을 조회할 때 admin-prod.default 를 사용한다.
monitoring	kube-prometheus-stack (Prometheus·Grafana)	애플리케이션을 다시 배포해도 모니터링 데이터를 계속 볼 수 있게 따로 둔다.

레거시 namespace `containerssh`, `cssh` 가 클러스터에 남아 있을 수 있다. 현재 서비스는 이 namespace를 쓰지 않는다. 리소스를 지우려면 실제 사용 중인지 먼저 확인한다.

2.3 관리할 때 쓰는 주소와 명령

대상	방법	비고
Grafana 대시보드	<code>http://210.94.179.18:30080</code>	admin 로그인. 비밀번호는 K8s Secret에서 kubectl로 조회한다
Swagger (config-server API)	<code>http://210.94.179.18:30082/apidocs/</code>	현재 API 목록을 볼 수 있다. 인증이 없으므로 <code>Try it out</code> 은 실제 계정·Pod·파일을 바꾼다. README의 9732 포트는 잘못된 값이며 NodePort는 30082다.
infra-mysql (pod_port_db)	<code>kubectl exec -it infra-mysql-0 -n ailab-infra -- mysql -u pod_port_user -p pod_port_db</code>	비밀번호는 K8s Secret에서 kubectl로 조회한다
config-server 로그	<code>kubectl logs -f deploy/containerssh-config-server -n ailab-infra</code>	승인·삭제 실패의 1차 확인 지점

Secret 값 조회는 이름을 찾은 뒤 jsonpath로 디코딩한다.

```
kubectl get secrets -n ailab-infra # Secret 이름 확인
kubectl get secret <이름> -n ailab-infra -o jsonpath='{.data.<키>}' | base64 -d
```

3. 작업 전 상태 확인

매일 한 번, 또는 배포·계정 작업을 하기 전에 아래 항목을 확인한다.

3.1 config-server가 떠 있는지

```
kubectl -n ailab-infra get deploy,sts,pod
kubectl -n ailab-infra get svc
kubectl -n ailab-infra get events --sort-by=.lastTimestamp
kubectl -n ailab-infra rollout status deploy/containersssh-config-server
```

다음 네 가지를 확인한다.

- config-server Deployment가 1/1 Ready이다.
- infra-mysql과 Redis 구성요소가 Ready이다.
- config-server Service가 30082 NodePort를 유지한다.
- 새 Warning 이벤트와 반복 재시작이 없다.

3.2 현재 배포 설정

비밀 값을 출력하지 않고 배포 구조를 확인한다.

```
kubectl -n ailab-infra get deploy containersssh-config-server -o custom-
columns=IMAGE:.spec.template.spec.containers[0].image,SA:.spec.template.spec.serviceAccountName,NODE:.spec.tem
plate.spec.nodeSelector
kubectl -n ailab-infra describe deploy containersssh-config-server
kubectl -n ailab-infra get svc containersssh-config-service
helm -n ailab-infra get manifest containersssh-config-server
```

현재 배포에는 아래 값이 들어가야 한다.

항목	확인할 값
replica	1
프로세스	gunicorn worker 4개, timeout 700초
ServiceAccount	config-server
config-server 배치	지정 control-plane 노드의 nodeSelector와 NoSchedule toleration
컨테이너 포트	8000
readiness	GET /health, 초기 15초, 10초 주기
liveness	별도 probe 없음
리소스	request 500m/512Mi, limit 2000m/1024Mi
API Service	NodePort 30082, Service 80 → container 8000
계정 저장소	NFS kube_share → /kube_share

항목	확인할 값
비밀 mount	NAS, FARM, FARM AD SSH Secret이 read-only
Kerberos 설정	krb5-conf ConfigMap → /etc/krb5.conf
image-store	pvc-image-store → /image-store

config-server 계정의 Kubernetes 권한도 확인한다.

```
kubectl auth can-i list pods -n ailab-infra --as=system:serviceaccount:ailab-infra:config-server
kubectl auth can-i create services -n ailab-infra --as=system:serviceaccount:ailab-infra:config-server
kubectl auth can-i list nodes --as=system:serviceaccount:ailab-infra:config-server
kubectl auth can-i list services --all-namespaces --as=system:serviceaccount:ailab-infra:config-server
```

마지막 명령은 다른 namespace에서 사용 중인 NodePort도 확인하는 권한이다. **no** 가 나오면 권한을 바로 바꾸지 말고, 현재 배포의 Role과 ClusterRole을 먼저 확인한다.

3.3 API 응답과 로그

```
config_server_url='운영_config-server_URL'

curl -fsS "$config_server_url/health"
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=30m
```

/health 는 config-server 프로세스만 확인한다. admin_be, MySQL, Redis, Prometheus, NAS, FARM SSH, AD 상태는 확인하지 않는다. /health 가 OK여도 승인이 실패하면 이 서비스들을 차례로 확인한다.

Pod 생성 로그에는 다음 태그가 차례로 나온다.

- **FETCH_USER_CONFIG** — admin_be 사용자 설정 조회
- **SELECT_NODE** — Prometheus 점수 기반 노드 선택
- **NODEPORT** — 포트 배정
- **WAIT_POD_READY** — Pod Ready 폴링
- **FARM SSH**, **FARM AD SSH** — 노드·AD 원격 작업

3.4 사용자 Pod와 접속 Service

```
kubectl -n ailab-infra get pod -l username
kubectl -n ailab-infra get svc -l app=ailab-nodeport
kubectl -n ailab-infra get pod -o wide
```

각 Service의 `pod_name` 라벨이 실제 Pod 이름과 같은지 확인한다. Error·Failed Pod나 연결 대상이 없는 Service를 발견해도 바로 지우지 않는다. 사용자 이름, 포트 배정 기록, 생성·삭제 로그를 먼저 확인한다(1절 5번).

특정 사용자 Pod의 실행 설정과 마운트를 확인할 때는 jsonpath로 필요한 필드만 뽑는다.

```
pod_name='점검할_ailab_Pod_이름'

kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{.spec.nodeName}{"\n"}{.spec.restartPolicy}{"\n"}{.spec.containers[0].image}{"\n"}{.spec.containers[0].resources}{"\n"}'
kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{range .spec.volumes[*]}{.name}{"\t"}{.hostPath.path}{"\t"}{.hostPath.type}{"\n"}{end}'
kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{range .spec.containers[0].volumeMounts[*]}{.name}{"\t"}{.mountPath}{"\t"}{.readOnly}{"\n"}{end}'
kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{range .spec.containers[0].env[*]}{.name}{"\n"}{end}'
```

사용자 Pod에서 다음 항목을 확인한다.

- FARM 홈이 `/home` 에 연결되어 있다. 노드에 마운트된 NFS 경로를 hostPath로 연결한다.
- Kerberos 활성 시 `/run/user/<uid>` 가 같은 경로로 연결되어 있다.
- GPU를 배정했다면 `/dev/nvidia<N>` 과 추가 NVIDIA 디바이스가 있다.
- **사용자 Pod에는 keytab, kube_share, image-store가 없어야 한다.** 이 셋이 보이면 잘못된 스펙이다.

현재 Pod 스펙에는 securityContext와 사용자 probe가 없다. 컨테이너 안의 UID/GID, SSH, Jupyter, noVNC 동작은 guest image의 entrypoint까지 확인해야 알 수 있다.

아래 명령은 중복된 NodePort만 출력한다. 아무것도 나오지 않아야 한다.

```
kubectl -n ailab-infra get svc -l app=ailab-nodeport -o jsonpath='{range .items[*].spec.ports[*]}{.nodePort}{"\n"}{end}' | sort -n | uniq -d
```

3.5 GPU와 NFS 상태

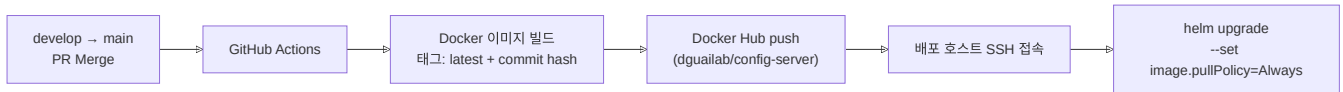
대상	방법
통합 대시보드 (Grafana)	<code>http://210.94.179.18:30080</code> admin 로그인
스토리지 응답성	Grafana 대시보드 <code>storage-latency-health</code> — NFS 장애 의심 시 가장 먼저 본다
Prometheus 직접 질의	<code>kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090</code>

Prometheus 값은 대시보드뿐 아니라 Pod를 올릴 노드를 고를 때도 쓴다. Prometheus 조회가 모두 실패하면 `create-pod` 도 실패한다. GPU 지표가 없을 때 노드를 고르는 방식은 **시스템 아키텍처**의 GPU 노드 선택 방식에 있다.

4. 배포와 롤백

4.1 배포가 진행되는 순서

배포 자동화는 GitHub Actions를 사용하며, **main** 브랜치에 **push 이벤트가 발생할 때만** 실행된다. 즉 **develop** → **main** PR이 Merge 되는 순간 운영 배포가 시작된다.



CI는 다음을 실행한다.

1. **config-server/Dockerfile** 로 이미지를 빌드한다.
2. **latest** 와 소스 버전을 나타내는 commit hash 태그 두 가지로 레지스트리에 푸시한다.
3. **config-server/Chart** 를 배포 호스트로 전달한다.
4. CI Secret의 NFS, NAS, FARM, AD, Realm 값을 Helm에 주입한다.
5. **ailab-infra** release의 이미지 태그를 commit hash로 갱신한다.

CI/CD용 GitHub Secrets는 **DOCKER_USERNAME**, **DOCKER_PASSWORD**, **K8S_HOST**, **K8S_USERNAME**, **K8S_PRIVATE_KEY**, **K8S_PORT** 이다. 현재 username이 toni 기준으로 등록되어 있어 관리자 변경 시 인수인계가 필요하다. 비밀 값은 repository 파일, 일반 운영 절차, 로그에 복사하지 않는다.

4.2 배포 전 확인

코드를 병합하기 전에 다음을 확인한다.

1. **다른 작업 진행 여부** — 다른 사람이 같은 클러스터에서 인프라 테스트를 하거나 Helm 값을 바꾸고 있으면, 변경 시점과 범위를 먼저 맞춘다.
2. **배포 브랜치** — main과 hotfix 계열 브랜치가 분기된 이력이 있어 Ready 대기·krb5 방식·마운트 구성이 브랜치마다 다르다. "지금 클러스터에 떠 있는 이미지가 어느 커밋인가"부터 확인한다.

명령으로는 다음을 확인한다.

```
helm lint config-server/Chart
git diff -- config-server .github/workflows/deploy-config-server.yaml
kubectl -n ailab-infra get deploy/containersssh-config-server -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
helm -n ailab-infra history containersssh-config-server
```

다음에 맞아야 한다.

- 변경 범위가 config-server와 chart 의도에 맞다.
- 이미지 태그가 commit hash인지 확인한다.
- chart가 참조하는 Secret과 ConfigMap 이름이 운영 네임스페이스에 존재한다(Helm 차트 레퍼런스 1.4절).
- FARM 노드 목록과 AD DC 목록의 이름이 실제 Kubernetes 노드 이름과 일치한다.

- `FARM_HOME_MOUNT_ROOT` 가 모든 후보 노드에서 동일한 FARM 홈을 가리킨다.

4.3 배포 뒤 확인

```
kubectl -n ailab-infra rollout status deploy/containersssh-config-server --timeout=5m
kubectl -n ailab-infra get pod -l app=containersssh-config-server -o wide
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=10m
kubectl -n ailab-infra get svc containersssh-config-service
```

로그에서 먼저 볼 오류는 다음과 같다.

- `ModuleNotFoundError` — requirements.txt 누락 또는 파일명 불일치이다.
- `WORKER TIMEOUT` — 초기 로딩 시간이 긴 경우이다 (Dockerfile 타임아웃 설정 확인).

이후 시험 사용자 또는 승인된 기존 사용자 한 명으로 실제 동작을 확인한다.

1. `/health` 를 호출한다.
2. 사용자 조회 API를 호출한다.
3. 시험 사용자 또는 승인된 기존 사용자로 Pod 생성 단계를 확인한다.
4. NodePort Service와 실제 접속을 확인한다.
5. 선택 노드의 Kerberos ccache와 FARM 홈 읽기·쓰기를 확인한다.
6. Pod 삭제 후 Service, 포트 예약, 노드 타이머가 정리되는지 확인한다.

4.4 롤백

배포 뒤 기존에 되던 기능이 동작하지 않으면 Helm release 이력을 보고 이전 버전으로 되돌린다.

```
helm -n ailab-infra history containersssh-config-server
helm -n ailab-infra rollback containersssh-config-server '정상_리버전'
kubectl -n ailab-infra rollout status deploy/containersssh-config-server --timeout=5m
```

롤백은 **config-server Deployment**만 이전 버전으로 되돌린다. 계정 파일, MySQL 포트 배정 기록, Kubernetes Service, FARM 노드의 keytab은 자동으로 이전 상태가 되지 않는다. 배포 중 실패한 요청이 있으면 해당 요청의 `stage`, `rollback`, 실제 리소스를 함께 확인한다.

5. 계정 관리

계정 정보는 `/kube_share` 의 passwd 파일에 기록한다(시스템 아키텍처의 계정 정보). 아래 API만 이 파일을 수정한다. passwd 파일을 직접 편집하지 않는다.

5.1 조회

```
config_server_url='운영_config-server_URL'
target_user='사용자명'

curl -fsS "$config_server_url/accounts/users"
curl -fsS "$config_server_url/accounts/users/$target_user"
```

계정 상세에서 사용자 이름, UID, GID, 홈, 셸을 확인한다. shadow와 keytab 내용은 조회하지 않는다(1절 4번).

5.2 생성

계정은 관리자 승인 뒤 admin_be가 만든다. 운영자가 직접 호출하는 것은 예외이며, 호출 전에는 같은 사용자가 없는지 확인한다.

```
curl -fsS -X PUT "$config_server_url/accounts/users" -H 'Content-Type: application/json' --data '{
  "name": "사용자명",
  "passwd_base64": "BASE64로_인코딩한_비밀번호",
  "gecos": "사용자 설명",
  "supplementary_groups": [
    {"name": "그룹명", "gid": 20050}
  ]
}'
```

성공 응답의 UID/GID를 기록하고(1절 2번) 다음 순서로 확인한다.

1. 사용자 조회 API
2. NAS 홈 디렉터리의 숫자 소유권과 0700
3. FARM AD 사용자와 <user>_gid
4. Kubernetes keytab Secret의 **존재 여부만** 확인 (내용은 보지 않는다)

```
target_user='사용자명'
kubectl -n ailab-infra get secret "krb5-keytab-$target_user" -o custom-
columns=NAME:.metadata.name,CREATED:.metadata.creationTimestamp
```

5.3 그룹

그룹 생성은 이름을 필수로 받고, GID를 생략하면 다음 사용 가능한 값을 할당한다.

```
curl -fsS -X PUT "$config_server_url/accounts/groups" -H 'Content-Type: application/json' --data '{"name":"그룹
명","members":["사용자명"]}'
```

기존 그룹에 사용자를 추가한다.

```
curl -fsS -X PUT "$config_server_url/accounts/users/사용자명/groups" -H 'Content-Type: application/json' --data '{"groups":["그룹명"]}'
```

사용자가 기본 그룹으로 쓰는 그룹은 삭제할 수 없다.

5.4 삭제

계정 삭제 전에 사용자의 Pod를 먼저 삭제한다(6.2절). Pod 이름과 선택 노드를 기록하고, Service와 NodePort 정리가 끝난 뒤 계정을 삭제한다. 순서를 지키지 않으면 주인 없는 Pod가 남는다.

```
target_user='사용자명'
curl -fsS -X DELETE "$config_server_url/accounts/users/$target_user"
```

삭제 뒤에는 다음을 확인한다.

- 계정 조회가 404를 반환한다.
- 사용자 Pod와 Service가 없다.
- keytab Secret이 없다.
- 모든 FARM 노드에서 사용자 타이머와 keytab이 없다.
- NAS 홈 디렉터리가 현재 보존·삭제 규칙에 맞게 처리되었다.

NAS 홈 삭제가 실패해도 계정 삭제는 계속 진행된다. **deleted** 응답 뒤에도 홈 디렉터리가 남을 수 있으므로 로그와 NAS 경로를 확인한다.

6. 사용자 Pod 관리

6.1 만들기

계정과 WAS 사용자 설정이 있는지 확인한 뒤 호출한다.

```
target_user='사용자명'

curl -fsS "$config_server_url/accounts/users/$target_user"
curl -fsS -X POST "$config_server_url/create-pod" -H 'Content-Type: application/json' --data '{"username":"'${target_user}'"}'
```

노드 선택, 포트 배정, krb5 배포, Pod 생성, Ready 대기를 차례로 처리하므로 응답까지 시간이 걸린다. 기다리는 동안 다른 터미널에서 진행 상태를 조회할 수 있다.

```
curl -fsS "$config_server_url/pods/$target_user/status"
```

성공 응답에서 `pod_name`, `node`, `ports` 를 기록하고(1절 2번), 실제 리소스를 확인한다.

```
pod_name='응답의_Pod_이름'

kubectl -n ailab-infra get pod "$pod_name" -o wide
kubectl -n ailab-infra get svc -l "pod_name=$pod_name"
kubectl -n ailab-infra describe pod "$pod_name"
```

Pod가 Ready이고, 응답의 각 포트에 해당하는 NodePort Service가 있으며, SSH·Jupyter·선택 기능에 실제로 접속 되면 생성이 끝난 것이다.

6.2 지우기

삭제 전 사용자에게 중단 사실을 확인하고 Pod 이름을 정확히 기록한다.

```
pod_name='삭제할_ailab_Pod_이름'

curl -fsS -X POST "$config_server_url/delete-pod" -H 'Content-Type: application/json' --data
'{"pod_name":"$pod_name"}'
```

응답의 `progress` 에서 각 단계의 완료 여부를 확인한다.

필드	의미
<code>servicesDeleted</code>	연결된 NodePort Service 삭제 완료
<code>nodeportsReleased</code>	MySQL 예약 해제 완료
<code>podDeleteRequested</code>	Kubernetes 삭제 요청 접수
<code>podDeleted</code>	실제 Pod 삭제 확인

삭제 후 실제로 사라졌는지 조회한다.

```
kubectl -n ailab-infra get pod "$pod_name"
kubectl -n ailab-infra get svc -l "pod_name=$pod_name"
```

Pod가 이미 없다는 성공 응답은 노드 이름을 읽지 못해 Kerberos 노드 정리를 건너뛰었을 수 있다. 이 경우 사용자와 마지막 노드를 기준으로 `kdc-setup` 운영 문서의 keytab, ccache, timer를 확인한다.

수동으로 지울 때 **kubectl** 로 Pod만 지우지 않는다. Service와 MySQL 배정 행이 남는다. 배정 정리(reconcile)는 같은 `pod_name` 의 Service가 하나라도 남아 있으면 그 Pod의 배정 행을 지우지 않는다. Service가 남은 상태에서는 이 배정이 계속 남는다(시스템 아키텍처의 NodePort 배정 방식).

6.3 사용자 Pod에 들어가 확인하기

어떤 경로든 Pod 이름 찾기부터 시작한다. 사용자 Pod 이름은 `ailab-<username>--<랜덤>` 형식이다.

```
kubectl get pods -n ailab-infra | grep <username>
```

(a) `kubectl exec` 로 확인하기. 사용자의 SSH 설정·비밀번호와 관계없이 root로 바로 들어간다. 컨테이너 프로세스, 마운트, 홈 권한을 확인할 때 쓴다.

```
kubectl exec -it ailab-user2100-1a2b3c4d -n ailab-infra -- bash
```

(b) 사용자가 쓰는 SSH 경로로 확인하기. 사용자가 접속하지 못한다는 문의는 이 경로로 확인한다. NodePort는 Service 목록이나 `infra-mysql` `nodeport_allocations` 에서 확인한다. 이 번호는 pfSense 전달 대상 확인에 쓰고, SSH 접속에는 사용자에게 안내한 외부 주소와 포트를 쓴다.

```
kubectl get svc -n ailab-infra | grep <username> # ssh 용도 Service의 NodePort 확인
ssh -p <사용자에게_안내한_외부_포트> <username>@<사용자에게_안내한_주소>
```

교내망에서는 되는데 외부에서만 접속하지 못하면 **시스템 아키텍처**의 외부 접근 경로를 보고 Service의 NodePort와 pfSense 전달 대상이 같은지 확인한다.

(c) 컨테이너에 들어갈 수 없을 때. Pod가 CrashLoop 등으로 `exec` 할 수 없으면 로그와 노드에서 확인한다.

```
kubectl logs ailab-user2100-1a2b3c4d -n ailab-infra # 컨테이너 stdout 로그
kubectl describe pod ailab-user2100-1a2b3c4d -n ailab-infra # 이벤트(FailedMount 등)
```

노드에 SSH로 들어가 컨테이너 런타임을 직접 보는 방법도 있다.

```
crictl ps | grep <username> # 해당 노드의 컨테이너 목록
crictl logs <container-id>
```

6.4 Pod가 죽었거나 노드에 문제가 있을 때

현재 config-server는 Pod가 죽은 것을 감지해 다른 노드에 자동으로 다시 만들지 않는다. 운영자가 다음 순서로 확인하고 복구한다.

1. 사용자 이름, 기존 Pod 이름, resource group, 노드, 이미지, NodePort를 기록한다.
2. Pod가 Failed인지, 노드가 NotReady인지, 애플리케이션만 응답하지 않는지 구분한다.
3. 같은 사용자의 다른 Running Pod가 없는지 확인한다.
4. FARM 홈과 AD·keytab Secret은 삭제하지 않는다. Pod를 다시 만들어도 계정 관련 파일은 그대로 써야 한다.
5. 장애 노드가 후보에서 다시 선택되지 않도록 WAS resource group 또는 노드 상태를 먼저 정리한다.
6. `/create-pod` 로 대체 Pod를 만들고 새 노드의 ccache, 홈, SSH·Jupyter를 확인한다.
7. 현재 구현은 새 NodePort를 할당하므로 **새 접속 정보를 사용자에게 전달한다**.

8. 대체 Pod가 정상인지 확인한 뒤에야 기존 Service, Pod, NodePort 배정 기록, 노드의 Kerberos 파일을 정리한다.

```
target_user='사용자명'

kubectl -n ailab-infra get pod -l "username=$target_user" -o wide
kubectl -n ailab-infra get svc -l "username=$target_user"
curl -fsS "$config_server_url/pods/$target_user/status"
```

자동으로 다시 만드는 기능이 없으므로 Failed Pod를 발견했다고 기존 Service부터 지우지 않는다.

6.5 Pod를 다른 노드로 옮길 때

`/migrate` 는 같은 resource group의 다른 노드가 충분히 여유 있을 때 새 Pod를 먼저 만드는 API다. 그러나 사용자 image-store 연결과 기존 NodePort 유지가 검증되지 않았으므로 **운영 사용자에게 일반적으로 실행하지 않는다.**

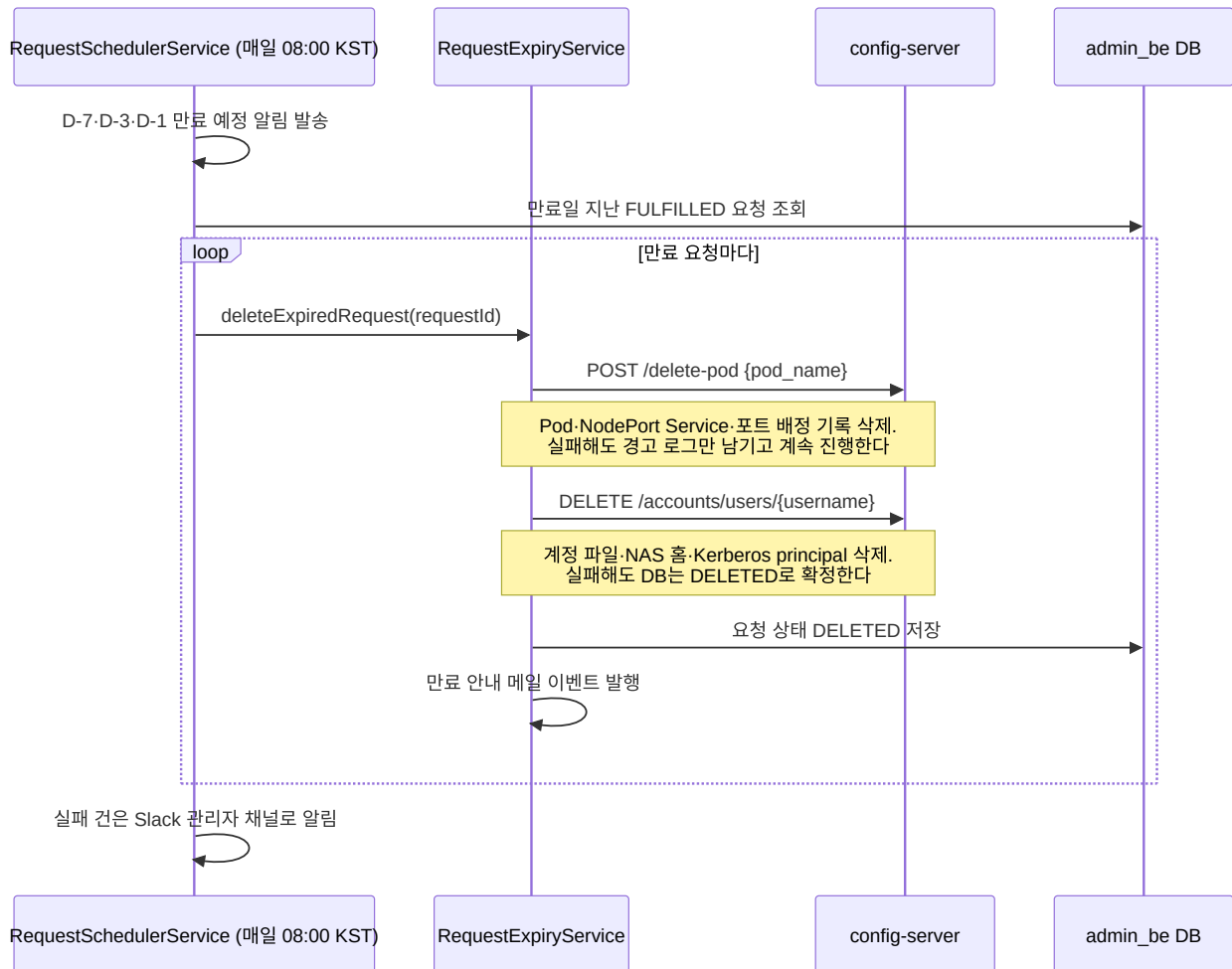
시험 환경에서 확인할 때는 다음을 모두 준비한다.

- 삭제해도 되는 시험 사용자와 재현 가능한 이미지
- 두 개 이상의 정상 GPU 후보 노드
- FARM 홈 백업 또는 시험 데이터
- 현재 Pod·Service·NodePort·선택 노드 기록
- 새 Pod 실패 시 기존 Pod를 유지하는 확인 절차
- 새·이전 노드의 Kerberos 자산 점검

새 Pod가 Ready여도 홈 데이터, 설치 패키지, SSH·Jupyter·noVNC, 포트 연속성, 이전 노드 정리까지 확인해야 한다.

6.6 만료된 사용자 정리

자원 회수는 admin_be의 일일 스케줄러가 자동 수행한다. `RequestSchedulerService` 가 매일 **08:00(KST)** 에 돌면서 만료 예정 알림을 보내고, 만료일이 지난 요청을 `RequestExpiryService` 에 넘겨 config-server API 두 개를 순서대로 호출한다. (사용자 비활성 스케줄러는 별도로 09:00에 돈다.)



만료 처리에서는 다음 두 가지를 확인한다.

1. **삭제할 때는 두 API를 모두 호출한다.** `POST /delete-pod` 는 Pod, Service, 포트만 지우고 `DELETE /accounts/users` 는 계정, 홈, principal만 지운다. 상세 명세는 [API 레퍼런스](#)를 참고한다.
2. **외부 정리가 실패해도 DB는 DELETED로 확정된다.** 인프라에 남은 자원이 있어도 admin_be 화면에는 삭제 완료로 보인다. 만료 처리가 있었던 날에는 8절에서 Pod, Service, 포트 기록을 확인한다. 과거에는 만료 처리에서 계정만 지우고 Pod·NodePort를 남기는 결함 T-KI-01이 있었고 26-07-06에 수정 배포했다. 만료 처리 경로는 아직 실제로 검증하지 않았다.

7. 문제가 생겼을 때

7.1 먼저 확인할 순서

문제가 생기면 API 응답부터 실제 리소스까지 다음 순서로 확인한다.

API HTTP 상태

- └ error stage / error code / rollback
 - └ config-server 로그
 - └ 연결된 서비스 (WAS·MySQL·Redis·Prometheus·NAS·FARM)
 - └ Kubernetes 실제 리소스
 - └ MySQL·FARM 노드 잔존 상태

같은 요청을 재시도하기 전에 부분 생성 여부를 확인한다. 특히 계정 생성과 Pod 생성은 여러 외부 저장소를 순차 갱신하므로, 중간 실패는 "일부만 만들어진" 상태를 남긴다.

7.2 증상별로 먼저 볼 곳

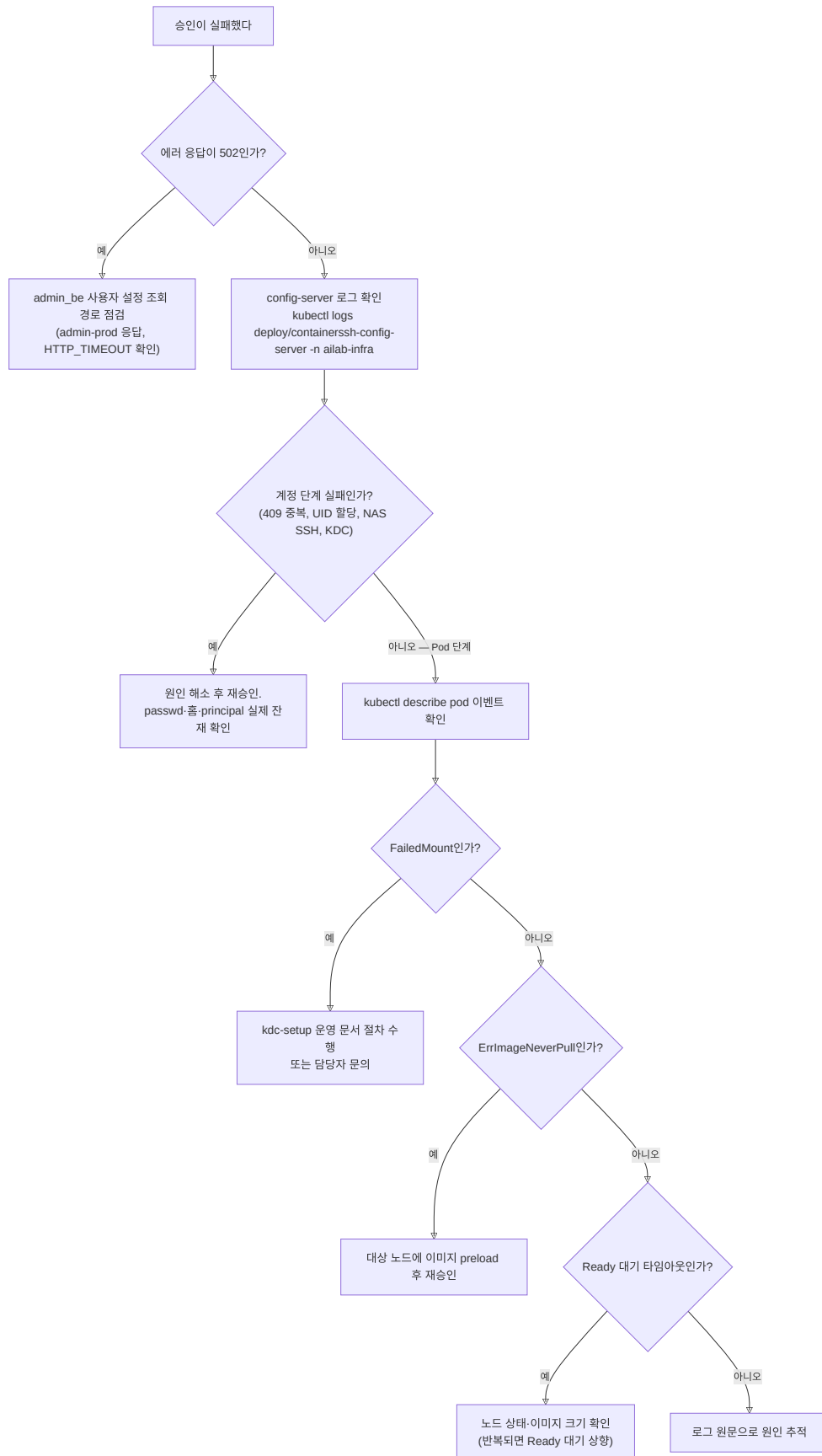
자주 생기는 문제와 먼저 볼 곳을 정리했다. 각 항목의 자세한 절차는 이어지는 절에 있다.

장애	증상	확인	해결
Pod Pending / ErrImageNeverPull	승인 후 Pod가 Pending, Ready 대기 초과로 승인 롤백	<code>kubectl -n ailab-infra describe pod <pod></code> 이벤트, 대상 노드 `cricctl images \	grep decs`
FailedMount (krb5)	<code>mount.nfs: Key has expired</code> 로 신규 Pod 홈 마운트 실패	<code>kubectl -n ailab-infra describe pod <pod></code> 의 FailedMount 이벤트	FARM NAS 홈 마운트에는 Kerberos 인증이 전제된다. kdc-setup 운영 문서 절차를 따르거나 담당자에게 문의한다
계정 생성 실패	승인이 계정 단계에서 실패(409, UID 할당 실패), PENDING 복귀	<code>kubectl -n ailab-infra logs deploy/containerssh-config-server, /kube_share/passwd</code> , NAS SSH 수동 확인	원인(중복 username, NAS SSH, KDC) 해소 후 재승인. 롤백 로그를 맹신하지 말고 passwd·홈·principal의 실제 잔재를 확인한다
NodePort 외부 접속 불가	내부(교내망)는 정상인데 외부 접속만 안 됨	<code>nodeport_allocations</code> 와 Service의 NodePort, pfSense 전달 대상, 사용자에게 안내한 외부 주소를 대조	pfSense 전달 대상이나 안내한 주소를 바로잡고 다시 접속 확인

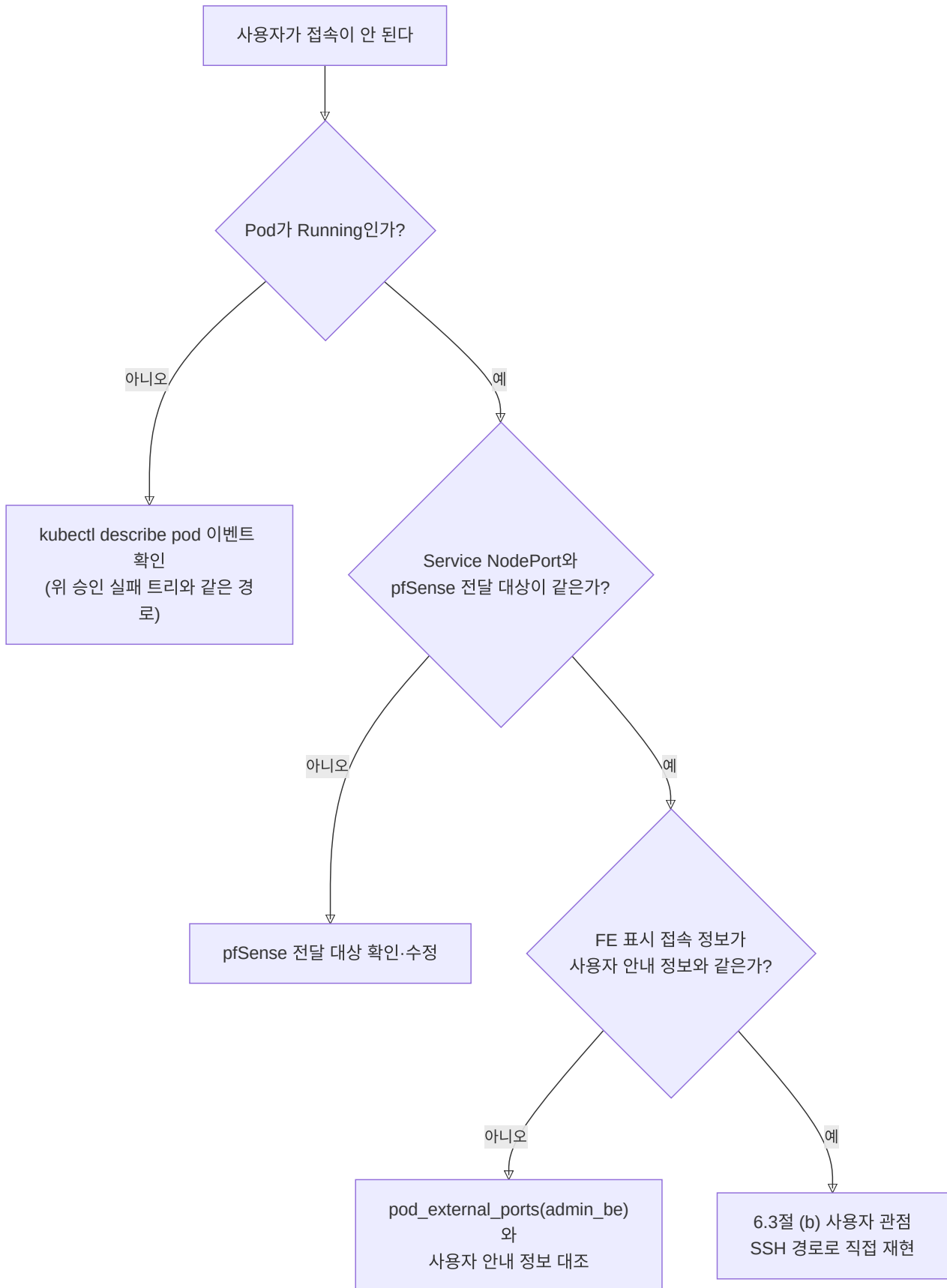
7.3 승인과 접속 문제 확인 순서

승인이 실패했을 때와 사용자가 접속하지 못할 때는 아래 순서로 확인한다.

트리 1 — 승인이 실패했다



트리 2 — 사용자가 접속이 안 된다



7.4 사용자 설정 조회 실패 (**FETCH_USER_CONFIG**)

config-server가 admin_be에서 사용자 설정을 가져오지 못한 경우다. 이때 승인은 되었지만 Pod 생성만 실패할 수 있다.

1. `admin-prod` Deployment와 Service 상태
2. 대상 사용자 승인·설정 존재 여부
3. config-server에서 admin_be Service DNS와 HTTP 응답
4. 응답 JSON의 image, gpu_nodes, additional_ports 형식
5. config-server의 3초 외부 HTTP 제한(`HTTP_TIMEOUT_SEC`) 안에 응답하는지

WAS가 HTTP 200 본문에 404 상태를 넣는 경우도 사용자 없음으로 처리된다.

7.5 노드 선택 실패 (`SELECT_NODE` , `NODE_LIST_FAILED`)

```
kubectl get node
kubectl get node -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=20m
```

- WAS가 내려준 후보 이름이 실제 Kubernetes 노드와 일치하는지 확인한다.
- `gpu_nodes` 가 비어 있을 때 사용할 Ready 상태의 일반 워커 노드가 있는지 확인한다.
- Prometheus가 각 후보의 GPU 지표를 반환하는지 확인한다. exporter가 죽은 노드는 0점으로 오인 선택될 수 있다(시스템 아키텍처의 GPU 노드 선택 방식).
- 선택된 노드에 FARM 홈과 Kerberos 관리 채널이 준비되어 있는지 확인한다.

7.6 NodePort 할당 또는 Service 생성 실패 (`NODEPORT` , `CREATE_NODEPORT_SERVICE`)

1. 요청 포트가 숫자이며 내부 포트가 중복되지 않는지 확인한다.
2. 30000-32767 범위의 실제 Service 사용 포트를 조회한다.
3. MySQL `nodeport_allocations` 의 같은 Pod·포트 예약을 조회한다(데이터베이스).
4. 오류 응답의 `nodeportsReleased` 와 `servicesDeleted` 를 확인한다.
5. Pod가 남아 있으면 연결된 Service 라벨을 대조한다.

포트 할당 기록과 Service가 다르다는 이유만으로 DB 행이나 Service를 바로 삭제하지 않는다. 사용자 접속 여부와 생성·삭제 로그를 먼저 확인한다.

7.7 Pod Ready 실패

```
kubectl -n ailab-infra describe pod "$pod_name"
kubectl -n ailab-infra logs "$pod_name" --all-containers
kubectl -n ailab-infra get events --field-selector "involvedObject.name=$pod_name"
```

증상별로 볼 곳이 다르다.

증상	우선 확인
<code>ImagePullBackOff</code> , <code>ErrImagePull</code> , <code>ErrImageNeverPull</code>	이미지 이름, 태그, 레지스트리 접근, 노드 캐시 (<code>crictl images</code>)

증상	우선 확인
CreateContainerConfigError	환경 변수, hostPath, Secret, ConfigMap
CrashLoopBackOff	guest image entrypoint, UID/GID, ccache, 셀 설정
Pending	직접 지정된 nodeName, 노드 Ready, hostPath, 디바이스
Ready 시간 초과	이미지 크기, 노드 I/O, 컨테이너 readiness 상태

생성 실패 응답이 Pod와 NodePort를 실제로 정리했는지 반드시 확인한다.

7.8 Kerberos 배포 또는 홈 접근 실패

deploying_krb5 단계에서 실패하면 다음을 확인한다.

1. 사용자 keytab Secret 메타데이터
2. 선택 노드와 FARM 노드 설정의 일치
3. ai lab-krb5 forced-command SSH 채널
4. 선택 노드의 decs-krb-refresh@<user> 로그
5. ccache 소유권과 FARM TGT
6. FARM 홈 마운트와 NAS 소유권

상세 명령과 머신 ccache 안전 규칙은 kdc-setup 운영 문서를 따른다.

7.9 MySQL 또는 Redis 장애

MySQL 장애는 NodePort 예약 실패로 Pod 생성을 중단시킨다. infra-mysql Pod, Service, PVC, 연결 제한과 config-server DB 오류를 확인한다.

Redis 장애는 진행 상태 기록만 누락시킨다 — 생성 자체는 성공할 수 있다. 생성 API와 Kubernetes 리소스가 성공했는데 상태 API가 unknown 이거나 오래된 단계라면 Redis 연결과 TTL을 확인한다. Redis 복구를 위해 정상 사용자 Pod를 삭제하지 않는다.

7.10 삭제가 중간에 멈췄을 때

오류 응답의 progress 와 실제 상태를 비교해 어느 단계까지 처리됐는지 확인한다.

실제 상태	다음 조치
Service만 삭제됨	사용자 접속 중단을 알리고 포트 할당 기록과 Pod 상태를 확인한다
Service·예약 삭제, Pod 남음	같은 Pod 이름으로 삭제 API 재시도 여부를 판단한다
Pod 삭제, 노드 Kerberos 파일 남음	Kerberos 재조정 작업 상태를 확인하고 대상 노드에서 정리한다
계정 삭제, NAS 홈 남음	데이터 보존 여부를 확인한 뒤 NAS 운영 절차로 처리한다

Kerberos 파일 정리가 실패한 경우에는 infra-mysql의 `krb5_cleanup_pending`도 확인한다. 이 테이블에 행이 있으면 30분마다 실행되는 CronJob이 다시 정리한다. CronJob도 실패하면 해당 사용자와 노드의 keytab, timer, ccache를 [kdc-setup 운영 문서](#) 순서대로 확인한다.

수동으로 정리하기 전에는 만료 스케줄러나 krb5 reconcile이 같은 파일을 다시 처리하지 않는지 확인한다.

8. 기록과 실제 상태 비교

MySQL·계정 파일의 기록이 Kubernetes·NAS·AD의 실제 상태와 같은지 정기적으로 확인한다. 만료 처리가 있었던 날에는 반드시 확인한다.

8.1 Pod·Service

```
kubectl -n ailab-infra get pod -l username -o custom-  
columns=NAME:.metadata.name,USER:.metadata.labels.username,NODE:.spec.nodeName,PHASE:.status.phase  
kubectl -n ailab-infra get svc -l app=ailab-nodeport -o custom-  
columns=NAME:.metadata.name,USER:.metadata.labels.username,POD:.metadata.labels.pod_name,NODEPORT:.spec.ports[0].nodePort
```

Pod가 없는 Service, Service가 없는 Running Pod, 같은 Pod에 중복으로 열린 목적 포트를 목록으로 남긴다. **이 단계에서는 지우지 않는다.** 기록과 실제 상태를 비교한 뒤에 처리 방법을 결정한다.

만료가 있었던 날의 간이 확인 명령이다.

```
kubectl get pods -n ailab-infra # 만료 사용자 Pod 잔류 여부  
kubectl get svc -n ailab-infra | grep ailab- # NodePort Service 잔류 여부
```

잔류가 보이면 수동 정리는 `POST /delete-pod` → `DELETE /accounts/users` 순서를 지킨다(6.2절, 5.4절).

8.2 계정·AD·NAS

시험 사용자 또는 문제가 있는 사용자에 대해 다음 값을 한 줄씩 비교한다. 값이 다른 행이 문제를 찾아볼 곳이다.

확인 위치	비교 값
config-server	사용자 이름, UID, GID
FARM AD	sAMAccountName, uidNumber, gidNumber, 전용 그룹
NAS	홈 경로, 숫자 소유권, 모드
선택 노드	ccache 소유 UID, principal
Pod	<code>id</code> , 홈 소유권, ccache 경로

9. 코드나 설정을 바꾼 뒤 확인할 것

코드나 설정을 바꾼 뒤 기존 기능이 깨지지 않았는지 확인한다.

9.1 API 또는 계정 로직

- 기존 응답 코드와 `stage`, `error_code`, `rollback` 형식을 유지한다.
- `passwd`·`group`·`shadow` 쓰기 순서와 실패했을 때 되돌리는 처리를 함께 시험한다.
- NAS, AD, Secret 중간 실패를 각각 시험한다.
- 사용자 이름과 그룹 입력 검증을 우회할 경로가 없는지 확인한다.

9.2 Pod 명세 또는 guest image

- UID/GID와 추가 그룹 환경 변수를 양쪽에서 같은 이름으로 사용한다.
- `/home` 과 `/run/user/<uid>` hostPath를 함께 검증한다.
- SSH 22, Jupyter 8888, noVNC 6080, 추가 포트 회귀 시험을 수행한다.
- GPU 디바이스와 nodeName 변경 뒤 다른 Pod가 같은 GPU 디바이스에 접근하지 않는지 확인한다.

9.3 Helm과 권한

- ServiceAccount의 namespaced 권한과 Node 조회 권한을 최소 범위로 유지한다.
- Secret 값은 values 파일과 로그에 남기지 않는다.
- 배포 후 실제 이미지 태그와 source 버전을 기록한다.

9.4 문제 확인에 쓰는 설정값

문제를 확인할 때 바꿔 보며 원인을 좁힐 수 있는 값들이다. 위치는 `config-server/main.py` 상단 `app.config` (코드 상수와 env), `config-server/Chart/values.yaml` (Helm 배포값) 두 곳이다.

키	위치	역할	디버깅 때 언제 바꾸나
<code>NAMESPACE</code> (<code>ai lab-infra</code>)	main.py 상수	모든 Pod·Service를 만드는 namespace	다른 리소스와 분리한 시험 클러스터에서만
<code>POD_READY_MAX_WAIT_SEC</code> (300초)	main.py	Ready 대기 초과 시 생성 롤백. 배포 브랜치에 따라 값이 다를 수 있음	큰 이미지 첫 기동으로 타임아웃 롤백이 반복될 때 상향
<code>imagePullPolicy</code>	main.py 사용자 Pod 스펙	<code>IfNotPresent</code> — 노드에 이미지가 없을 때만 내려받는다	<code>ErrImagePull</code> 원인을 확인할 때 이미지 이름·태그와 노드의 이미지 목록을 먼저 확인한다
<code>PROM_URL</code>	main.py 상수	노드 선택용 Prometheus 주소	모니터링 스택 주소·포트가 바뀌었을 때
<code>WAS_URL_TEMPLATE</code>	main.py 상수	admin_be 사용자 설정 조회 주소	admin_be 배포 위치(namespace·Service명) 변경 시

키	위치	역할	디버깅 때 언제 바꾸나
<code>HTTP_TIMEOUT_SEC</code> (3.0)	main.py 상수	admin_be·Prometheus 호출 제한 시간	느린 응답으로 502가 날 때 원인을 확인할 때
<code>BASE_ETC_DIR</code> (<code>/kube_share</code>)	main.py 상수	계정 파일(passwd 등) 루트 경로	운영에서는 바꾸지 않는다. 시험 환경에서만 변경한다.
<code>NFS_SERVER</code> / <code>NFS_USER_SHARE_PATH</code>	env ← values.yaml <code>nfs.*</code>	사용자 홈 NFS 마운트 원천	NAS 이전·export 경로 변경 시
<code>KRB5_REALM</code>	env ← values.yaml <code>krb5.*</code>	비우면 Kerberos 설정을 만들지 않는다	KDC 설정을 관리하는 사람과 확인한 뒤 변경
<code>db.*</code> 블록	values.yaml	NodePort 배정 DB(infra-mysql) 접속점	포트 배정 실패 시 접속 대상이 맞는지 확인
<code>redis.*</code> 블록	values.yaml	사용자 이미지 저장/로드 상태 Redis	이미지 상태 조회 실패 시 접속 대상 확인
<code>namespace</code> / <code>nodeSelector</code>	values.yaml	배포 namespace·config-server 배치 노드 고정	고정 노드 점검으로 임시 이전 시 (tolerations 짝 확인)

변경 방법은 재배포 1회이다. values.yaml 값은 `helm upgrade` 로 반영하고, main.py 상수는 이미지 재빌드가 필요하므로 CI/CD(main 머지) 경로를 쓴다.

```
helm upgrade --install containerssh-config-server ./Chart -n ailab-infra
```

10. 하지 않는 작업

- shadow, keytab, SSH 개인키, DB 비밀번호를 화면·로그·티켓·일반 운영 문서에 붙이지 않는다. 관리자 전용 문서 외에는 원문을 남기지 않는다.
- 사용자 확인 없이 Running Pod와 홈을 삭제하지 않는다.
- MySQL NodePort 행만 지우거나 Service만 지우지 않는다. 기록과 실제 리소스가 더 어긋날 수 있다.
- 대체된 ContainerSSH·사용자 PVC chart를 현재 복구 경로로 재배포하지 않는다.
- 레거시 **restart 스크립트**(`restart_auth.sh`·`restart_containerssh.sh`·`restart_pvc.sh`)를 운영 배포에 사용하지 않는다. 전부 레거시 namespace(`containerssh`·`cssh`)를 향하던 것으로 레거시 차트 정리와 함께 레포에서 제거됐다. 과거 체크아웃에 남아 있어도 쓰지 않는다. 운영 서비스는 전부 `ailab-infra`에 있으며, config-server 재배포는 4.1절 CI/CD 또는 `config-server/Makefile`(`make deploy`) 경로를 쓴다.
- 이미지 저장·마이그레이션 경로를 검증 없이 운영 사용자에게 실행하지 않는다(6.5절).
- D-state(디스크 I/O에 걸려 죽지도 살지도 못하는 프로세스 상태)가 있는 FARM 노드에서 NFS 복구를 반복하지 않는다.

11. 참고 문서

- 계정·Pod·포트가 만들어지는 방식 — 시스템 아키텍처
- 배경 용어 — 기초 개념
- API 전체 명세 — API 레퍼런스
- 배포값·Secret 이름·차트 구조 — Helm 차트 레퍼런스
- NodePort 배정 테이블 — 데이터베이스
- FARM AD·Kerberos·NFS 장애 — kdc-setup 운영 문서

API 레퍼런스

이 문서에서 다루는 내용

- config-server HTTP API의 엔드포인트, 호출 주체, 입력, 응답
- 어떤 API가 실제 admin_be 에서 호출되고 어떤 API는 수동 확인용인지
- Pod·계정 생성과 삭제 요청에서 주의할 오류 형식

빠른 이동

필요할 때	문서
승인 뒤 내부 처리 순서를 먼저 본다	시스템 아키텍처
운영 중 오류 응답을 해석한다	운영 매뉴얼
Kerberos 준비 실패를 본다	kdc-setup 운영

config-server의 HTTP API 명세다. main.py 의 Flask route와 /accounts blueprint를 합쳐 총 12개 엔드포인트를 제공한다.

현재 배포된 API 목록은 Swagger UI http://210.94.179.18:30082/apidocs/ 에서도 확인할 수 있다.

1. 공통 사항

항목	값
Base URL (운영)	http://210.94.179.18:30082 (namespace ailab-infra , Service containersssh-config-service)

항목	값
요청/응답 형식	JSON (Content-Type: application/json)
인증	없음 — 승인된 운영망 밖으로 노출하지 않는다. 현재 Helm 차트에는 NetworkPolicy가 없으므로 클러스터의 별도 접근 제어가 있는지 확인한다.
에러 응답 형태	Pod 계열은 infra_error() 포맷(step, error, detail, progress 등), 계정 계열은 {"error": "..."} 단순 포맷

호출 주체는 admin_be(Spring Boot WAS)의 서비스 클래스이다. admin_be 실코드 기준으로 확인한 매핑이다.

admin_be 서비스	호출 엔드포인트	시점
AdminRequestCommandService	PUT /accounts/users	관리자 승인 시
PodService	POST /create-pod, POST /delete-pod	승인 시 / 만료 또는 실패를 되돌리는 삭제 시
UbuntuAccountService	DELETE /accounts/users/{username}	만료 또는 실패를 되돌리는 삭제 시
GroupService	PUT /accounts/groups	그룹 생성 시
(호출처 없음 — 미확인)	GET /pods/{username}/status, POST /migrate, GET /accounts/*, DELETE /accounts/groups, PUT /accounts/users/{u}/groups	admin_be 코드에서 호출처가 발견되지 않았다. 운영자 수동 호출·실험용으로 보인다

2. GET /health

항목	값
메서드·경로	GET /health
호출 주체	헬스체크(서버 생존 확인). admin_be 서비스 호출은 없다

입력. 없다.

처리 순서. 즉시 응답한다. 외부 의존성(DB, Kubernetes)을 검사하지 않으므로, 200이 와도 DB 연결 실패 상태일 수 있다.

성공 응답. 200 본문 "OK" (문자열).

대표 실패. 없다(무조건 200).

3. POST /create-pod

항목	값
메서드·경로	POST /create-pod
호출 주체	admin_be PodService (승인 트랜잭션의 마지막 단계)

입력.

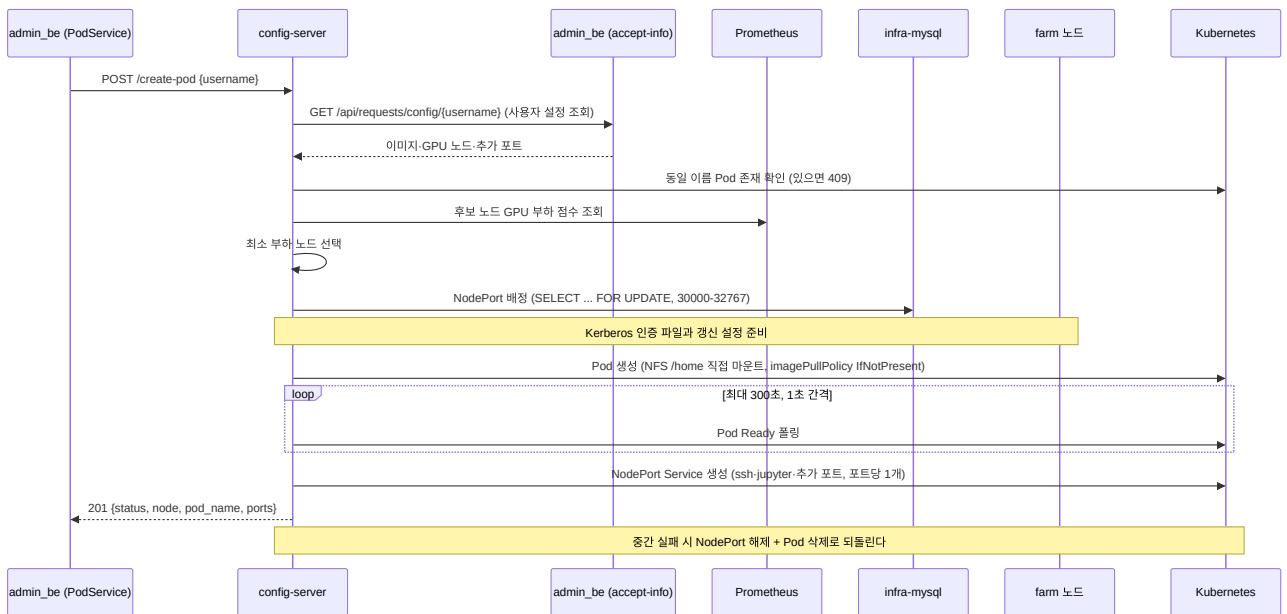
```
{"username": "user2100"}
```

`username` 하나만 받는다. 이미지, GPU 노드, 추가 포트는 config-server가 admin_be에 다시 요청해 가져온다.

처리 순서.

1. `username` 없으면 400을 반환한다.
2. admin_be에 GET `http://admin-prod.default/api/requests/config/{username}` 을 요청해 사용자 설정(이미지, GPU 노드 목록, 추가 포트)을 받아온다. 통신 실패나 JSON 파싱 실패는 502, 사용자가 없으면 404이다.
3. Pod 이름을 `ai lab-<username>-<랜덤>` 형식으로 생성한다.
4. 같은 이름의 Pod가 이미 있으면 409를 반환한다. 같은 username이라도 이름이 다른 Pod는 막지 않으므로, 사용자 단위 중복 호출은 admin_be가 막아야 한다.
5. 후보 노드 목록을 만든다. admin_be가 준 `gpu_nodes` 를 사용하고, 비어 있으면 Ready 상태의 워커 노드 전체를 사용한다.
6. Prometheus(서버 지표를 수집하는 모니터링 시스템)에 각 후보 노드의 GPU 부하를 물어 **가장 한가한 노드**를 고른다.
7. Pod 스펙을 조립한다(`build_pod_spec`). 이 단계 안에서 아래 일이 함께 일어난다.
 - `/kube_share` 계정 파일 확인 — **passwd에 사용자가 없으면 실패**한다(계정 생성이 선행 조건).
 - 저장된 사용자 이미지(`/image-store/images/user-<username>.tar`)가 있으면 로드하고, 없으면 WAS가 준 base 이미지를 쓴다.
 - 기본 포트 22(ssh), 8888(jupyter), 추가 포트에 NodePort를 배정한다. DB에서 30000~32767 중 빈 번호를 고르고 `SELECT ... FOR UPDATE` 로 동시에 같은 번호를 고르지 못하게 한다. 클러스터 전체 Service 조회에 실패하면 DB 기록만으로 배정하므로, 이미 다른 Service가 쓰는 포트를 고르면 뒤의 Service 생성이 실패할 수 있다.
 - 추가 포트에 내부 포트 6080 이 있거나 용도가 `novnc` , `vnc` 이면 Pod 환경 변수에 `ENABLE_VNC=true` 를 넣는다.
 - Kerberos 설정이 켜져 있으면 선택된 FARM 노드에 인증 파일과 갱신 설정을 준비한다. 실패하면 여기서 중단된다. 점검과 복구는 [kdc-setup 운영 문서](#)를 따른다.
 - NFS user-share를 `/home` 에 직접 마운트하도록 볼륨을 구성한다. 사용자 Pod의 `imagePullPolicy` 는 `IfNotPresent` 이므로 노드에 이미지가 없을 때만 레지스트리에서 가져온다.
1. Kubernetes에 Pod를 생성한다.
2. 최대 300초 동안 1초 간격으로 Pod가 Ready 상태가 되기를 기다린다.
3. Ready가 되면 포트마다 NodePort Service(`ai lab-<username>-<용도>-<외부포트>`)를 생성한다.

4. 성공 응답을 반환한다. 8~10단계 어디서든 실패하면 배정한 NodePort 해제 + 만든 Pod 삭제(+Service 삭제)로, 그때까지 만든 것을 되돌린다.



성공 응답. 201

```
{
  "status": "created",
  "node": "farm5",
  "pod_name": "ailab-user2100-1a2b3c4d",
  "ports": [
    {"internal_port": 22, "external_port": 30000, "usage_purpose": "ssh"},
    {"internal_port": 8888, "external_port": 30001, "usage_purpose": "jupyter"}
  ]
}
```

대표 실패.

상 태	의미
400	<code>username</code> 누락, 또는 스펙 조립 단계의 검증 실패(알 수 없는 노드, passwd에 사용자 없음 등)
404	WAS에 해당 사용자가 없음 (<code>USER_CONFIG_NOT_FOUND</code>)
409	동일 이름 Pod가 이미 존재 (<code>POD_ALREADY_EXISTS</code>)
502	admin_be 사용자 설정 조회 실패 — 통신 오류·비정상 응답 (<code>USER_CONFIG_FETCH_FAILED</code>)
500	노드 선택·NodePort 배정·Kerberos 준비·Pod 생성·Ready 대기·Service 생성 실패. 응답의 <code>rollback</code> 필드로 어디까지 되돌렸는지 알 수 있다

4. GET /pods/<username>/status

항목	값
메서드·경로	GET /pods/<username>/status
호출 주체	생성 진행 상황을 조회하는 운영자·클라이언트용 API. admin_be 호출처는 미확인

POST /create-pod 는 Pod 준비가 끝날 때까지 최대 300초 동안 기다리는 동기 API이다. 이 API는 생성 요청이 끝나기 전에도 사용자별 진행 상황을 가볍게 조회한다.

입력. 경로의 `username`.

성공 응답. 200

```
{
  "username": "user2100",
  "stage": "waiting_ready",
  "message": "이미지 pull / 컨테이너 기동 대기 중",
  "updated_at": "2026-07-22T01:23:45+00:00"
}
```

stage 는 `unknown`, `started`, `selecting_node`, `building_pod_spec`, `allocating_nodeport`, `deploying_krb5`, `creating_pod`, `waiting_ready`, `creating_services`, `ready`, `failed` 중 하나이다. 생성 이력이 없으면 stage 는 `unknown` 이고 `updated_at` 은 없다. `failed` 의 `message` 에는 보안을 위해 상세 예외나 Kubernetes 오류 원문을 넣지 않으므로, 원인은 config-server 로그에서 확인한다.

대표 실패. 상태 저장소 조회에 실패하면 500 (`POD_STATUS_LOOKUP_FAILED`)을 반환한다.

5. POST /delete-pod

항목	값
메서드·경로	POST /delete-pod
호출 주체	admin_be PodService (만료 스케줄러, 실패를 되돌리는 삭제)

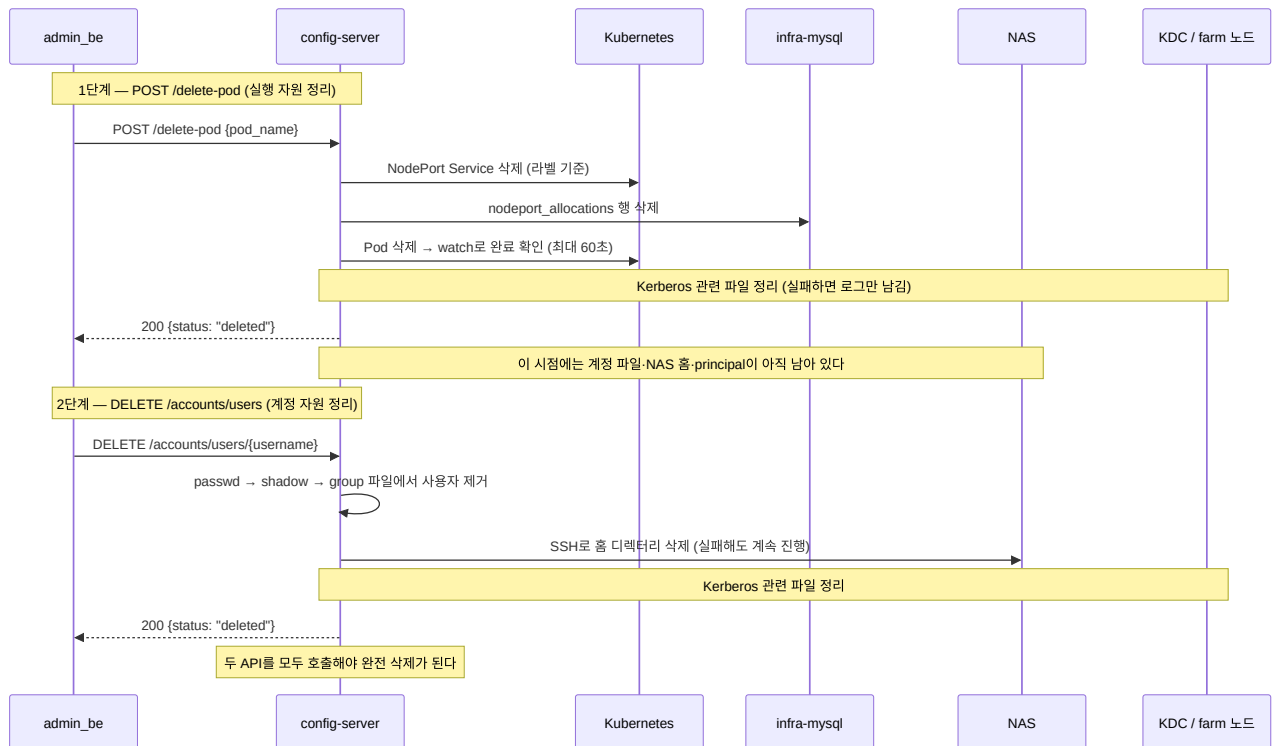
입력.

```
{"pod_name": "ai lab-user2100-1a2b3c4d"}
```

처리 순서.

1. `pod_name` 없으면 400, `ai lab-` prefix가 아니면 400(`INVALID_POD_NAME`)을 반환한다. 이름에서 username을 파싱한다.
2. 해당 Pod의 NodePort Service들을 라벨(`app=ai lab-nodeport, pod_name=<pod>`) 기준으로 삭제한다.
3. `nodeport_allocations` 테이블에서 해당 Pod의 포트 배정 기록 행을 삭제한다.
4. Kubernetes Pod를 삭제하고, watch(쿠버네티스 이벤트 스트림 구독)로 **실제 삭제 완료**를 최대 60초 기다린다. Pod가 원래 없었으면(404) `already_absent: true` 로 200을 반환한다.
5. Kerberos 관련 파일을 정리한다. 이 정리가 실패해도 Pod 삭제 결과는 실패로 바꾸지 않고 로그만 남긴다.

이 API는 Pod, Service, NodePort DB 행만 정리한다. 계정 파일(passwd 등), NAS 홈, Kerberos principal은 그대로 남고, 아래 10절의 `DELETE /accounts/users` 가 정리한다. 두 API 중 하나만 호출하면 Pod 관련 자원이나 계정 관련 파일이 남는다.



성공 응답. 200

```

{
  "status": "deleted",
  "pod_name": "ai lab-user2100-1a2b3c4d",
  "progress": {
    "servicesDeleted": true,
    "nodeportsReleased": true,
    "podDeleteRequested": true,
    "podDeleted": true
  }
}

```

`progress` 는 롤백 결과가 아니라 **각 단계의 완료 여부**이다. Pod가 원래 없던 경우 `already_absent: true` 가 추가된다.

대표 실패.

상태	의미
400	<code>pod_name</code> 누락, 또는 <code>ai lab-</code> prefix가 아닌 이름
500	Service 삭제·NodePort 해제·Pod 삭제 실패, 또는 60초 내 삭제 미완료(<code>POD_DELETE_TIMEOUT</code>)

6. POST /migrate

항목	값
메서드·경로	<code>POST /migrate</code>
호출 주체	admin_be 호출처 없음 — 운영자 수동·실험용 (미확인)

입력.

```
{
  "username": "user2100",
  "nodes": ["farm5", "farm6"],
  "min_improvement_ratio": 0.2
}
```

`username` 과 `nodes` (후보 노드 목록)가 필수이고, `min_improvement_ratio` (이만큼은 좋아져야 옮긴다는 개선 문턱, 기본 0.2)는 선택이다.

처리 순서.

- 사용자별 락 파일(`/tmp/migrate-<username>.lock`)을 잡아 같은 사용자의 동시 마이그레이션을 막는다.
- 후보 노드 이름을 실제 클러스터 노드 이름으로 정규화한다. 모르는 노드가 있으면 400이다.
- 실행 중인 사용자 Pod를 찾는다. 없으면 404이다. 현재 노드가 후보 목록에 없으면 400이다.
- 현재 노드를 뺀 후보가 없으면 `skipped` (`no_candidate_node`)로 200을 반환한다.
- Prometheus GPU 점수를 현재 노드와 후보들에 대해 계산한다. 최고 후보가 `min_improvement_ratio` 만큼 충분히 좋지 않으면 `skipped` (`no_significant_improvement`)로 200을 반환한다.
- WAS에서 사용자 설정을 다시 조회한다.
- 기존 Pod 안에서 이미지 commit/save를 실행해 사용자 상태를 image-store에 저장한다. 실패하면 500이다.
- 새 Pod 이름을 만들고, 새 노드에 Pod를 생성한 뒤 Ready를 최대 60초 기다리고 NodePort Service를 만든다. 실패하면 새 Pod와 새로 배정한 포트를 정리하고 500이다.
- 새 Pod가 완전히 성공한 뒤에야 기존 Pod의 Service·포트 배정·Pod를 삭제한다.

성공 응답. `200`

```
{
  "status": "migrated",
  "from": "farm5",
  "to": "farm6",
  "new_pod": "ailab-user2100-9z8y7x6w",
  "ports": [{"internal_port": 22, "external_port": 30005, "usage_purpose": "ssh"}]
}
```

건너뛴 경우에도 200이며 `{"status": "skipped", "reason": ...}` 형태이다.

대표 실패.

상태	의미
400	<code>username</code> / <code>nodes</code> 누락, 알 수 없는 노드, 현재 노드가 후보 목록에 없음
404	실행 중인 Pod 없음
500	이미지 commit 실패(<code>image_commit_failed</code>), 새 Pod 기동 실패, Service 생성 실패

7. GET /accounts/users

항목	값
메서드 · 경로	<code>GET /accounts/users</code>
호출 주체	admin_be 호출처 없음 — 운영자 점검용 (미확인)

입력. 없다.

처리 순서. NFS 계정 파일 `/kube_share/passwd` (리눅스 계정 목록 파일)를 읽어 전체 사용자를 반환한다.

성공 응답. 200

```
{
  "users": [
    {
      "name": "user2100",
      "uid": 20001,
      "gid": 20001,
      "gecos": "GPU User",
      "home": "/home/user2100",
      "shell": "/bin/bash"
    }
  ]
}
```

대표 실패. 파일 읽기 예외 시 500(`{"error": ...}`).

8. GET /accounts/users/<username>

항목	값
메서드·경로	GET /accounts/users/<username>
호출 주체	admin_be 호출처 없음 — 운영자 점검용 (미확인)

입력. 경로의 `username`.

처리 순서.

- passwd에서 사용자를 찾는다. 없으면 404이다.
- group 파일을 읽어 기본 그룹과 추가 그룹을 함께 반환한다.

성공 응답. 200

```
{
  "user": {"name": "user2100", "uid": 20001, "gid": 20001, "gecos": "", "home": "/home/user2100", "shell":
"/bin/bash"},
  "groups": [
    {"name": "user2100", "gid": 20001, "type": "primary"},
    {"name": "developers", "gid": 20005, "type": "supplementary"}
  ]
}
```

대표 실패.

상태	의미
404	사용자 없음
500	파일 읽기 예외

9. PUT /accounts/users

항목	값
메서드·경로	PUT /accounts/users
호출 주체	admin_be AdminRequestCommandService (승인 시)

입력.

```
{
  "name": "user2100",
  "passwd_base64": "cGFzc3dvcmQ=",
  "gecos": "GPU User",
  "primary_group_name": "user2100",
  "supplementary_groups": [{"name": "ailab", "gid": 20005}]
}
```

필수는 `name`, `passwd_base64` (Base64로 감싼 평문 비밀번호)이다. 나머지는 선택이다.

이전 문서에는 `uid`, `gid`, `passwd_sha512` 가 필수라고 적혀 있지만 현재 코드와 다르다. UID/GID는 서버가 빈 번호를 골라 정하고, 비밀번호 해시(SHA-512 crypt)도 서버가 만든다.

처리 순서.

1. 필수 필드와 `supplementary_groups` 형식(`{name, gid}` 목록)을 검증한다. `passwd_base64` 디코딩에 실패하면 400이다.
2. `/kube_share` 계정 파일 구조가 비어 있으면 `base_etc/` 템플릿으로 초기화한다.
3. `passwd` 파일을 잠가 다른 요청이 동시에 같은 UID를 고르지 못하게 한다. NFS 파일에는 잠금을 걸지 않고 로컬 `/tmp` 잠금 파일을 쓴다. 사용자가 이미 있으면 409를 반환한다. 없으면 20000 이상이고 `/home/` 을 쓰는 사용자 중 가장 큰 UID 다음 번호를 찾아 `passwd`에 추가한다. GID는 UID와 같은 값이다.
4. `group` 파일에 기본 그룹을 만들고, 요청에 든 추가 그룹에는 사용자를 구성원으로 넣는다. 실패하면 `passwd`에 쓴 내용도 되돌리고 500을 반환한다.
5. `shadow`(암호 해시 파일)에 SHA-512 crypt 해시를 기록한다. 실패하면 롤백 후 500이다.
6. `SUDO_ALLOWED_COMMANDS` 설정이 있으면 사용자별 `sudoers`(sudo 사용 권한을 정의하는 파일)를 0440 권한으로 만든다.
7. NAS(`192.168.2.30:6954`)에 SSH로 접속해 홈 디렉터리를 만든다(`mkdir`, `chown`, `chmod 700`). NFS의 `root_squash` 설정 때문에 Pod는 홈 디렉터리를 직접 만들 수 없다. 실패하면 계정 파일을 되돌리고 500을 반환한다.
8. Kerberos 설정이 켜져 있으면 사용자 `principal`과 `keytab Secret`을 만든다. 실패하면 홈 디렉터리를 지우고 계정 파일을 되돌린 뒤 500을 반환한다. 점검과 복구는 [kdc-setup 운영 문서](#)를 따른다.

성공 응답. 201

```
{
  "status": "created",
  "user": {"name": "user2100", "uid": 20001, "gid": 20001, "home": "/home/user2100", "shell": "/bin/bash"},
  "group": {"name": "user2100", "gid": 20001},
  "supplementary_groups": [{"name": "ailab", "gid": 20005}],
  "sudoers": "/kube_share/sudoers.d/user2100"
}
```

대표 실패.

상 태	의미
400	필수 필드 누락, <code>passwd_base64</code> 디코딩 실패, <code>supplementary_groups</code> 형식 오류
409	사용자 이미 존재
500	<code>group/shadow/sudoers</code> 기록 실패, NAS SSH 실패(<code>NAS_SSH_FAILED</code>), KDC 실패(<code>KDC_FAILED</code>) — 모두 계정 파일을 되돌린 뒤 반환한다

10. DELETE /accounts/users/<username>

항목	값
메서드 · 경로	<code>DELETE /accounts/users/<username></code>
호출 주체	<code>admin_be UbuntuAccountService</code> (만료, 실패를 되돌리는 삭제)

입력. 경로의 `username`.

처리 순서.

1. `passwd`에서 사용자를 제거한다. 없으면 404이다.
2. `shadow`에서 해당 행을 제거한다.
3. `group` 파일을 정리한다 — 모든 그룹의 멤버 목록에서 사용자를 빼고, 이 사용자가 쓰던 그룹(명시적 멤버였거나 `primary GID` 그룹)이 비게 되면 그룹 자체를 삭제한다.
4. NAS 홈 디렉터리를 SSH로 삭제한다. **실패해도 경고 로그만 남기고 계속 진행한다**(계정 파일은 이미 지워진 상태).
5. Kerberos 관련 파일을 정리한다. 이 단계가 실패하면 오류를 기록하고 다음 작업을 계속한다.

`POST /delete-pod`도 함께 호출해야 Pod와 계정이 모두 정리된다(5절의 시퀀스 다이어그램 참조). 이 API만 호출하면 Pod와 NodePort가 남고, `delete-pod`만 호출하면 계정, 홈, `principal`이 남는다.

성공 응답. `200 — {"status": "deleted", "user": "user2100"}`

대표 실패.

상태	의미
404	사용자 없음
500	Kerberos 정리 등 처리 중 예외 (NAS 홈 삭제 실패는 500이 아니라 경고 후 진행)

11. PUT /accounts/groups

항목	값
메서드·경로	PUT /accounts/groups
호출 주체	admin_be GroupService

입력.

```
{"name": "developers", "gid": 20005, "members": ["user2100", "user2101"]}
```

필수는 **name** 이다. **gid** 는 생략하면 group 파일 기준으로 빈 번호를 골라 자동 배정(20000 이상)하고, **members** 는 생략 가능하다.

처리 순서.

1. **gid** 타입을 검증한다(정수 또는 정수 문자열만 허용, 아니면 400).
2. **members** 에 적힌 사용자가 passwd에 전부 존재하는지 확인한다. 없는 사용자가 있으면 400이다.
3. group 파일에 락을 걸고 — 같은 이름이 있으면 409, 지정한 gid가 이미 쓰이면 409, 아니면 새 그룹 행을 추가한다.

성공 응답. 201 — {"group": {"name": "developers", "gid": 20005}}

대표 실패.

상태	의미
400	name 누락, gid 타입 오류, 존재하지 않는 멤버
409	그룹 이름 또는 gid 중복

12. DELETE /accounts/groups/<groupname>

항목	값
메서드·경로	DELETE /accounts/groups/<groupname>
호출 주체	admin_be 호출처 없음 (미확인)

입력. 경로의 **groupname** .

처리 순서.

1. group 파일에서 그룹을 찾는다. 없으면 404이다.

- 이 그룹을 기본 그룹으로 쓰는 사용자가 passwd에 있으면 삭제를 거부하고 400을 반환한다. 오류 응답에는 해당 사용자 이름이 들어간다.
- 그룹 행을 삭제한다.

성공 응답. 200 — {"status": "deleted", "group": "developers", "gid": 20005}

대표 실패.

상태	의미
400	기본 그룹으로 사용 중
404	그룹 없음

13. PUT /accounts/users/<username>/groups

항목	값
메서드 · 경로	PUT /accounts/users/<username>/groups
호출 주체	admin_be 호출처 없음 (미확인)

입력.

```
{"groups": ["developers", "ai-lab"]}
```

처리 순서.

- groups 목록이 비어 있으면 400이다.
- 사용자가 passwd에 없으면 404이다.
- 지정한 그룹들의 멤버 목록에 사용자를 추가한다(이미 있으면 그대로 둔다).
- 요청한 그룹 중 존재하지 않는 것이 있으면 404를 반환한다.

성공 응답. 200 — {"status": "updated", "user": "user2100", "groups": ["ai-lab", "developers"]}

대표 실패.

상태	의미
400	groups 누락
404	사용자 없음, 또는 존재하지 않는 그룹 포함

14. 호출할 때 확인할 점

- 위 12개 API는 요청자를 확인하지 않는다. 승인된 운영망 밖에서 호출할 수 없게 해야 한다. 현재 Helm 차트는 NetworkPolicy를 만들지 않으므로, 네트워크 접근 제어는 클러스터 설정에서 따로 확인한다.
- `create-pod`를 같은 사용자에게 여러 번 호출하면 Pod와 Service가 여러 개 만들어질 수 있다. 사용자 단위 중복 호출은 admin_be가 막아야 한다.
- `POST /delete-pod` (Pod, Service, 포트)와 `DELETE /accounts/users` (계정, 홈, principal)를 모두 호출해야 관련 자원이 남지 않는다.
- 8888 포트도 다른 포트처럼 NodePort Service로 열린다. 게스트 이미지의 Jupyter에 인증이 없으면 외부에서 접속할 수 있으므로 내부망에서만 접근하게 해야 한다.
- 만료 스케줄러가 이 API들을 어떤 순서로 부르는지는 [운영 가이드](#)의 만료 흐름 절을 참고한다.

Helm 차트 레퍼런스

이 문서에서 다루는 내용

- config-server Helm 차트가 어떤 리소스를 만드는지
- `values.yaml`과 템플릿이 Deployment, Service, RBAC, CronJob에 어떻게 반영되는지
- 배포 값을 바꾸기 전에 어디를 확인해야 하는지

빠른 이동

필요할 때	문서
Kubernetes 기본 용어를 먼저 본다	기초 개념
배포와 장애 대응 순서를 본다	운영 매뉴얼
Kerberos 설정 값의 위치를 본다	kdc-setup 설정

쿠버네티스 기본 용어(Pod, Service, PVC, namespace 등)는 [기초 개념](#)을 참고한다.

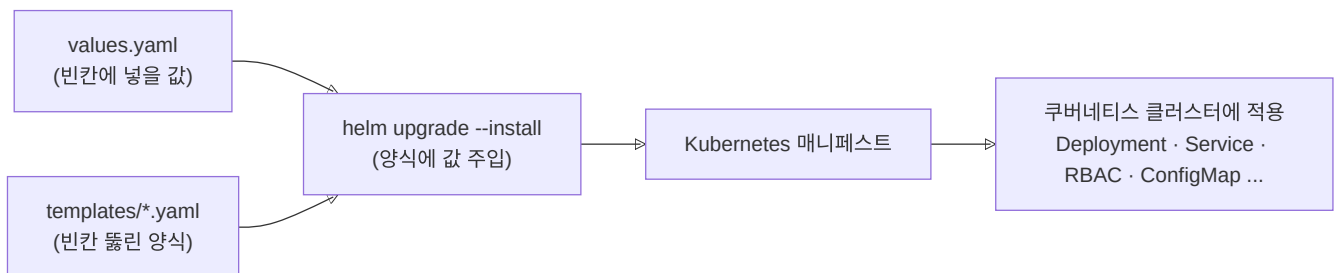
0. Helm 기본 개념

0.1 왜 Helm을 쓰는가

쿠버네티스에 무언가를 배포하려면 YAML 파일(매니페스트)을 여러 장 써야 한다. config-server 하나만 해도 Deployment, Service, 권한(RBAC), 설정(ConfigMap) 등 파일이 많다. 문제는 이 파일들 곳곳에 **배포 환경마다 달라지는 값**(이미지 태그, NFS 서버 주소, 포트 번호)이 박혀 있다는 점이다. 값이 바뀔 때마다 여러 파일을 뒤져 고치는 것은

실수하기 좋다.

- **템플릿(templates/)** — 매니페스트의 구조를 적어 둔 양식이다. 달라지는 자리는 `{{ .Values.image.tag }}` 처럼 빈칸으로 뚫어 둔다.
- **values.yaml** — 그 빈칸에 채워 넣을 값들의 모음이다. "이번 배포의 설정 전체"가 이 파일 한 장에 모인다.
- **차트(Chart)** — 템플릿 묶음 + values.yaml + 메타데이터(Chart.yaml)를 합친 패키지이다.
- **릴리스(release)** — 차트를 클러스터에 설치한 결과물의 이름이다. 같은 차트를 이름만 다르게 여러 번 설치할 수도 있다. 이 시스템의 릴리스 이름은 `containerssh-config-server` 이다.



배포 명령(`helm upgrade`)을 실행하면 Helm이 값을 양식에 넣어 Kubernetes 매니페스트를 만들고 클러스터에 적용한다. 예를 들어 NFS 서버 주소가 바뀌면 템플릿을 고치지 않고 values.yaml의 값을 바꾸면 된다.

0.2 자주 쓰는 명령

```
# 배포(설치+업데이트 겸용) — ./Chart 디렉터리의 차트를 릴리스에 반영한다
helm upgrade --install containerssh-config-server ./Chart -n ailab-infra

# 새 Pod가 정상적으로 떴는지 확인한다
kubectl rollout status deploy/containerssh-config-server -n ailab-infra

# 잘못됐으면 직전 배포 상태로 되돌린다 (Helm이 릴리스 이력을 기억한다)
helm rollback containerssh-config-server -n ailab-infra
```

`upgrade --install` 은 "릴리스가 없으면 설치하고, 있으면 갱신하라"는 겸용 명령이라 설치와 업데이트를 구분해 외울 필요가 없다. `rollback` 이 가능한 이유는 Helm이 릴리스마다 배포 이력을 버전으로 쌓아 두기 때문이다.

0.3 --set 으로 바꾼 값

`helm upgrade ... --set image.tag=abc123` 처럼 명령어 뒤에 값을 직접 붙이면 values.yaml을 고치지 않고 그 배포 한 번에만 값을 바꿀 수 있다. 다음 배포 때 values.yaml의 값으로 돌아가므로, 계속 유지할 변경은 values.yaml이나 별도 운영 values 파일에 기록한다.

1. Chart/ — config-server 차트 (현행)

config-server 배포용 Helm 차트이다. 위치는 저장소의 `config-server/Chart/` 이다.

1.1 이 차트가 클러스터에 만드는 것

리소스	이름	왜 필요한가
Deployment	containerssh-config-server	config-server 컨테이너 1개(replicas: 1)를 실행한다. 15초 뒤부터 10초마다 /health를 확인해 Ready 상태를 판단한다.
Service (NodePort)	containerssh-config-service	config-server의 고정 접속 주소이다. 내부 80 → 컨테이너 8000(gunicorn), 외부 NodePort 30082
ServiceAccount	config-server	config-server가 Kubernetes API를 호출할 때 사용하는 계정이다.
Role + RoleBinding	config-server-role / config-server-rolebinding	ailab-infra namespace 안에서 Pod(exec-log 하위 리소스 포함)·Service·PVC·Secret을 만들고 지을 권한을 위 ServiceAccount에 붙인다. 사용자 Pod 생성·삭제뿐 아니라 Pod 안에서 명령 실행(exec)·로그 조회까지 이 권한으로 이뤄진다
ClusterRole + Binding	...-node-reader	노드 목록 조회 권한이다. 노드는 namespace에 속하지 않는(cluster-scoped) 리소스라서 Role로는 못 주고 ClusterRole이 따로 필요하다
ConfigMap	krb5-conf	Kerberos 클라이언트 설정(krb5.conf)을 담아 config-server Pod에 주입한다
CronJob	...-krb5-reconcile	30분마다 reconcile_krb5.py를 실행한다. krb5_cleanup_pending에 남은 실패 건을 다시 정리하고, FARM 노드의 keytab 목록과 nodeport_allocations를 비교해 Pod가 없는 사용자의 keytab도 지운다.

1.2 파일 구성

파일	역할
Chart.yaml	차트 이름(containerssh-config-server)·버전 메타데이터이다
values.yaml	배포마다 달라지는 설정값 모음이다(1.3절)
templates/deployment.yaml	Deployment 양식이다 — 이미지·환경변수·마운트가 여기서 조립된다. values의 값 대부분이 컨테이너 환경변수로 주입된다
templates/service.yaml	NodePort Service 양식이다
templates/rbac.yaml	Role/RoleBinding + ClusterRole/ClusterRoleBinding 양식이다
templates/serviceaccount.yaml	ServiceAccount 양식이다
templates/configmap.yaml	krb5.conf ConfigMap 양식이다
templates/cronjob-krb5-reconcile.yaml	30분마다 Kerberos 정리 재시도와 남은 keytab 확인을 실행하는 CronJob 설정이다.
templates/_helpers.tpl	리소스 이름 조립 헬퍼이다 — 릴리스 이름을 그대로 리소스 이름으로 쓴다

값은 `values.yaml` 에서 `deployment.yaml` 템플릿의 `env` 를 거쳐 컨테이너 환경 변수로 들어간다. `main.py`, `utils.py` 는 이 환경 변수를 읽는다. 예를 들어 `nas.ssh.host` 는 `NAS_SSH_HOST` 가 되어 NAS SSH 접속에 사용된다.

1.3 values.yaml에 넣는 값

아래 기본값은 저장소 `values.yaml` 을 기준으로 적었다. `""` (빈 값)인 키는 실제 배포 때 별도 values 파일이나 `--set` 으로 넣는다. 저장소에는 주소·계정 같은 민감값을 두지 않는다.

image — 무엇을 배포하나

CI/CD가 새 이미지를 push한 뒤 이 값 기준으로 배포된다.

키	의미	기본값
<code>image.repository</code>	운영 이미지 저장소	<code>dguai lab/config-server</code>
<code>image.tag</code>	이미지 태그 (CI가 latest + commit hash 이중 태깅)	<code>latest</code>
<code>image.pullPolicy</code>	이미지를 매번 새로 받을지 정하는 정책	<code>Always</code>

service — 어떻게 접속하나

admin_be와 관리자가 이 포트로 API를 호출한다.

키	의미	기본값
<code>service.type</code>	Service 타입	<code>NodePort</code>
<code>service.port</code>	클러스터 내부 포트	<code>80</code>
<code>service.targetPort</code>	컨테이너 포트 (gunicorn)	<code>8000</code>
<code>service.nodePort</code>	외부 노출 NodePort	<code>30082</code> (README의 9732는 오기, kubectl 실측)

resources — config-server 자신의 몫

사용자 Pod의 리소스가 아니라 **관리 서버 본인**이 쓸 CPU/메모리이다. `requests`는 스케줄링 시 보장받는 최소치, `limits`는 상한이다.

키	의미	기본값
<code>resources.requests</code>	CPU/메모리 요청	<code>500m</code> / <code>512Mi</code>
<code>resources.limits</code>	CPU/메모리 상한	<code>2000m</code> / <code>1024Mi</code>

nfs / nas / farm — 연결할 서버 설정

사용자 홈 NFS 주소, NAS SSH 접속 정보, keytab을 배포할 FARM 노드 목록을 정한다.

키	의미	기본값
<code>nfs.server</code>	NFS 서버 주소	<code>""</code> (배포 시 주입)

키	의미	기본값
<code>nfs.userSharePath</code>	사용자 홈 루트 export 경로	""
<code>nfs.kubeSharePath</code>	계정 파일(<code>/kube_share</code>) export 경로 — config-server Pod 자신이 이 NFS를 마운트한다	""
<code>nas.ssh.host</code> / <code>port</code> / <code>user</code> / <code>keyPath</code>	NAS 홈 생성용 SSH 접속 정보 (키 실체는 Secret, 1.4절)	""
<code>farm.ssh.user</code> / <code>keyPath</code> / <code>nodes</code>	farm 노드 keytab 배포용 SSH 정보와 대상 노드 목록	"" / []
<code>farm.adSsh.user</code> / <code>keyPath</code> / <code>nodes</code>	AD/DC(krb5 principal 발급) 노드용 SSH 정보	"" / []
<code>farm.homeMountRoot</code>	노드 호스트에 마운트된 홈 루트 — 사용자 Pod <code>/home</code> 의 hostPath 원천이다 (시스템 아키텍처의 사용자 Pod 그림)	""

security / krb5 — 권한과 인증

키	의미	기본값
<code>security.sudoAllowedCommands</code>	사용자별 sudoers whitelist 명령 목록. 비어 있으면 sudoers 파일 자체를 만들지 않는다	""
<code>krb5.realm</code>	Kerberos realm(인증 도메인 이름) — 운영은 <code>AILAB.DGU</code> 이다	""
<code>krb5.kdcHost</code>	KDC(티켓 발급 서버) 호스트. 기본 <code>values.yaml</code>과 현재 CI 배포 명령에는 이 값이 없다. ConfigMap 템플릿은 이 값을 읽으므로, 현재 배포 결과의 <code>krb5.conf</code> 에는 <code>kdc</code> 와 <code>admin_server</code> 값이 비어 있다. 이 ConfigMap을 실제 Kerberos 클라이언트 설정으로 쓸 계획이면 <code>values</code> 와 CI에 주입 경로를 추가해야 한다.	(키 없음)

`krb5.realm`이 비어 있으면 config-server는 사용자 keytab과 ccache를 준비하지 않는다. NAS의 Kerberos NFS 설정에 이 값이 필요하면 새 Pod가 홈에 접근하지 못할 수 있다. 실제 NFS mount 옵션은 FARM 노드에서 확인한다.

db / redis — 상태 저장소

NodePort 배정 기록은 MySQL에, 이미지 저장·로드 상태는 Redis에 둔다(시스템 아키텍처의 NodePort 배정 방식).

키	의미	기본값
<code>db.host</code>	infra-mysql 호스트	<code>infra-mysql</code>
<code>db.name</code>	데이터베이스명	<code>pod_port_db</code>
<code>db.user</code>	DB 사용자	<code>pod_port_user</code>
<code>redis.host</code>	이미지 상태용 Redis	<code>redis-bg-master.ailab-infra.svc.cluster.local</code>
<code>redis.port</code>	Redis 포트	<code>6379</code>

DB 비밀번호는 values에 없다 — Secret `config-server-db-secret` 에서 주입된다(1.4절).

배치 관련 — config-server가 어디에 뜨나

키	의미	기본값
<code>namespace / config.namespace</code>	배포 대상 namespace. ConfigMap 템플릿은 <code>.Values.namespace</code> 를 읽고 나머지 템플릿은 <code>.Values.config.namespace</code> 를 읽는다. 두 값은 항상 같게 둔다.	<code>ailab-infra</code>
<code>nodeSelector</code>	config-server 배포 노드 고정	<code>kubernetes.io/hostname: csid-dgu-desktop</code>
<code>tolerations</code>	control-plane의 NoSchedule taint 허용. taint는 노드에 걸린 "여기 배치 금지" 표식이고, toleration은 그 금지를 통과하는 허가증이다. 고정 대상 노드가 control-plane이라 nodeSelector와 반드시 짝으로 있어야 Pod가 뜬다	<code>node-role.kubernetes.io/control-plane</code>
<code>cleanup.image</code>	현재 템플릿에서는 읽지 않는 값이다. 지워도 배포 결과는 달라지지 않는다.	<code>config-server:latest</code>

1.4 미리 만들어 둘 Secret

민감값(비밀번호, SSH 개인키)은 values.yaml에 넣지 않고 쿠버네티스 Secret으로 관리한다. Secret은 차트가 만들어 주지 않으므로 **클러스터에 미리 존재해야** 하며, 없으면 Pod가 뜨지 못한다.

Secret 이름	내용	소비처
<code>config-server-db-secret</code>	DB 비밀번호 (key: <code>password</code>)	Deployment-CronJob의 <code>DB_PASSWORD</code> 환경변수
<code>nas-ssh-key</code>	NAS 접속용 SSH 개인키	<code>/etc/nas-ssh</code> 에 read-only 마운트
<code>farm-ssh-key</code>	farm 노드 접속용 SSH 개인키	<code>/etc/farm-ssh</code> 에 read-only 마운트
<code>farm-ad-ssh-key</code>	AD/DC 노드 접속용 SSH 개인키	<code>/etc/farm-ad-ssh</code> 에 read-only 마운트

이 외에 config-server는 사용자별 `krb5-keytab-<user>` Secret을 만든다. `rbac.yaml` 의 Secret 생성 권한은 이 작업에 필요하다.

1.5 값을 바꿀 때

먼저 위치부터 잡는다. `config-server/` 는 GitHub 저장소(CSID-DGU/admin_infra) 안의 폴더이다. 저장소를 내려받아(`git clone https://github.com/CSID-DGU/admin_infra.git`) 그 안의 `config-server/` 로 이동한 상태에서 아래를 실행한다. 단, ②~④는 클러스터에 연결된 관리 서버(kubectl과 helm이 설정된 컴퓨터)에서만 동작한다. 개인 노트북에서 clone만 받아서는 ①(파일 수정)까지만 가능하다.

기본 `values.yaml` 에는 NFS·NAS·FARM·Kerberos의 실제 값이 비어 있다. 운영에서 수동으로 `helm upgrade` 를 실행할 때는 CI가 넣는 것과 같은 별도 values 파일 또는 `--set` 값을 함께 사용한다. 빈 기본값만으로 아래 명령을 실행하지 않는다.

```
# ① values.yaml에서 값을 수정한다
vi Chart/values.yaml

# ② 릴리스에 반영한다 (재배포)
helm upgrade --install containerssh-config-server ./Chart -n ailab-infra

# ③ 새 Pod가 정상적으로 떴는지 확인한다
kubectl rollout status deploy/containerssh-config-server -n ailab-infra

# ④ 잘못됐으면 직전 배포 상태로 되돌린다
helm rollback containerssh-config-server -n ailab-infra
```

반영 뒤에는 [운영 매뉴얼](#)의 배포 확인 절차를 따른다. 일회성 실험에는 `--set` 을 쓸 수 있지만, 계속 유지할 값은 0.3절처럼 파일에 남긴다.

2. pvc-image-chart/ — 이미지 저장소 (현행)

사용자 컨테이너 이미지를 tar로 보관하는 공용 PVC `pvc-image-store` 를 배포하는 작은 차트이다. **현행 구조에서 이 PVC를 마운트하는 것은 config-server뿐이다**(`/image-store` — 사용자별 이미지 저장/로드 경로는 `/image-store/images/user-<username>.tar`). 과거에는 게스트 Pod에도 붙였지만 마운트 실패 문제로 사용자 Pod 스펙에서는 제거됐다. 사용자별 홈 PVC가 폐기된 뒤에도 **image-store만 PVC로 남았다**([시스템 아키텍처](#)의 홈 디렉터리와 이미지 저장소).

키/장치	값	의미
<code>imageStore.size</code>	<code>500Gi</code>	PVC 요청 용량
<code>storageClass</code>	<code>nfs-nas-v3-expandable</code>	NFS CSI 기반 StorageClass — 어떤 방식으로 저장소를 만들지 정한 템플릿
<code>accessModes</code>	<code>ReadWriteMany</code>	여러 Pod가 동시에 마운트할 수 있는 모드. 현행 소비자는 config-server이지만, 이미지 저장소를 공유 스토리지로 유지하는 계약을 나타낸다
<code>helm.sh/resource-policy: keep</code>	<code>annotation</code>	릴리스를 지워도 PVC는 남긴다. 차트를 실수로 삭제해도 이미지 저장소 PVC가 함께 지워지지 않게 한다.

3. pvc-chart/ (폐기)

사용자/그룹별 홈 PVC를 동적 생성하던 Helm 차트(`containerssh-pvc-chart`)의 잔재이다. 개별 사용자 PVC 방식은 26-07-02에 폐기되어 현행 흐름에서는 사용하지 않는다(현행은 NFS 직접 마운트 — [시스템 아키텍처](#)의 홈 디렉터리와 이미지 저장소 참조).

이 디렉터리는 저장소에 남아 있지만 현재 배포에는 쓰지 않는다. `Chart.yaml`, `values.yaml`, `README.md`, StorageClass 매니페스트 두 장(`sc-nfs-nas-v3-resizable.yaml`, `old-sc.yaml`)은 이전 클러스터 상태를 확인할 때만 본다. 새로 배포하거나 복구할 때는 이 차트를 쓰지 않고 NFS 직접 마운트와 `pvc-image-chart/`를 사용한다.

4. infra-sql/ (현행)

config-server의 NodePort 배정 기록 등 인프라 상태를 저장하는 MySQL을 배포하는 **일반 매니페스트**다. Helm 차트가 아니라 `kubectl apply`로 YAML을 적용한다.

파일	역할
<code>nfs-mysql.yaml</code>	MySQL용 NFS CSI StorageClass <code>sc-mysql</code> 을 만든다 (NFS server <code>192.168.2.30</code> , share <code>/volume1/share</code> , reclaimPolicy <code>Retain</code> — PVC를 지워도 데이터 볼륨을 삭제하지 않고 남긴다)
<code>infra-mysql.yaml</code>	<code>ailab-infra</code> namespace에 <code>infra-mysql</code> StatefulSet(고정 이름과 저장소를 유지하는 Pod 관리 객체, 저장소 10Gi) + ClusterIP Service(클러스터 내부 전용 접속 주소)를 생성한다

DB에는 StatefulSet을 쓴다. Deployment와 달리 Pod가 다시 만들어져도 같은 이름과 저장소(PVC)를 다시 사용하므로 데이터가 이어진다. Service는 ClusterIP(클러스터 내부 전용)다. NodePort 배정 기록 DB를 클러스터 밖에 열지 않는다.

infra-mysql은 `nodeport_allocations`와 `krb5_cleanup_pending` 테이블을 자동으로 만들지 않는다. DB를 다시 만들면 두 테이블의 생성 SQL을 직접 실행해야 한다. 스키마는 [데이터베이스](#)를 참고한다.